

IBM EXPERT LABS

IBM Bob + Maximo MCP Hands-on Lab Guide

Maximo Application Suite 9.2 MCP Enablement

Guide type	Student hands-on lab guide
Lab sequence	Lab 01 through Lab 10
Platform	IBM Maximo Application Suite 9.2
Publication	IBM Expert Labs

About This Guide

This guide provides a progressive learning path for configuring IBM Bob, connecting to Maximo through Model Context Protocol, enabling Maximo tools, creating custom MCP tools, and completing an end-to-end maintenance workflow. Each lab remains a distinct module so that individual labs can be revised and republished without restructuring the complete guide.

Learning Path

Lab	Title	Primary outcome
01	Install Bob and Open the Workspace	Install IBM Bob, sign in, and open the workspace.
02	Connect IBM Maximo to Bob via MCP	Configure the Maximo MCP server.
03	Set Up Bob to Talk to Maximo	Configure the Maximo workspace rule file.
04	Discover Maximo MCP Tools	Review and invoke available tools.
05	Create a Maximo Service Request	Create a Service Request through Bob.
06	Talk to Maximo in Plain Language	Perform Maximo queries and actions.
07	Enable Object Structure Tools	Activate native Object Structure tools.
08	Create a Work Order MCP Tool	Build and test a Work Order tool.
09	Create an Automation Script MCP Tool	Build the Inspection Failure Advisor.
10	Capstone Maintenance Workflow	Complete an end-to-end workflow.

How to Use This Guide

Complete the labs in numerical order. Later labs depend on the configuration, tools, and records created in earlier labs. Record identifiers when instructed because later exercises may reference them.

Document Conventions

Prompt: Text entered into IBM Bob or another client.

Code: JSON, Jython, configuration, or schema content entered exactly as shown.

Expected result: The observable result used to verify successful completion.

Important: A condition that must be completed before continuing.

Table of Contents

About This Guide	2
Learning Path	2
How to Use This Guide	2
Document Conventions	2
Table of Contents	3
Lab 1: Install Bob and Open the Workspace	7
What You Will Do	7
Before You Start	7
Exercise 1.1 — Find Out What Kind of Mac You Have (Mac users only)	7
Exercise 1.2 — Download IBM Bob	8
Exercise 1.3 — Install Bob	8
On a Mac	8
On Windows	8
Exercise 1.4 — Sign-up for IBM Trial	8
Exercise 1.5 — Launch IBM Bob	9
On a Mac	9
On Windows	9
Exercise 1.6 — Sign In	9
Exercise 1.7 — Open the Lab Workspace Folder	10
Exercise 1.8 — Verify Everything Is Working	10
Exercise 1.9 — Install PDF Viewer	10
.....	12
Troubleshooting	12
Summary	13
Lab 2: Connecting IBM Maximo to Bob via MCP	14
Prerequisites	14
What Is MCP?	14
Exercise 2.1 — Obtain the MCP URL (Check Assigned-Environments folder)	14
.....	16
Exercise 2.2 — MAS Manage Login	16
Exercise 2.3 — Create Maximo Manage API Key	18

.....	19
Exercise 2.4 — Locate the Workspace MCP Config File	19
Exercise 2.5 — Add the Maximo Server Entry	21
Lab Values	22
Exercise 2.6 — Save the File	22
Exercise 2.7 — Verify the Connection in the Bob MCP Panel	22
Security Note	23
Reference: MCP Config Field Summary	23
Summary	24
Lab 3: Bob Rules	25
What You Will Do	25
Exercise 3.1 Update the RESTAPI and MCP Service URL's	25
Lab 4: Bob Navigation	28
What You Will Do	28
Exercise 4.1 - Navigation	28
Lab 5: Discover What Maximo Tools Are Available	31
What You Will Do	31
Exercise 5.1 — Ask Bob What Tools Are Available	31
Exercise 5.2 — Use the Tool	32
What This Means	32
Why Only One Tool?	33
Troubleshooting	33
Summary	33
Lab 6: Ask Bob to Create a Maximo Service Request	34
Exercise 6.1 — Report the Issue	34
Exercise 6.2 — Try Your Own Request	35
What Just Happened?	35
If Something Goes Wrong	35
Summary	35
Lab 7: Talking to Maximo in Plain Language	36
What You Will Do	36
Exercise 7.1 — Look Up an Asset	36
Exercise 7.2 — Explore a Station's Assets	37
Exercise 7.3 — Filter the Results	38
Exercise 7.4 — Update an Asset Status	38

Exercise 7.5 — Look Up Work Orders for an Asset	39
Exercise 7.6 — Find a Job Plan	39
Exercise 7.7 — Create a Work Order from a Job Plan and Link It to an SR	41
What Just Happened?	41
Key Takeaway	42
Troubleshooting	42
Summary	42
Lab 8: Enabling Maximo Object Structure Tools for Bob	43
What You Will Do	43
Exercise 8.1 — Enable the MXAPISR-CREATSR Object Structure Tool	43
Exercise 8.2 — Activate the Tool	44
Exercise 8.3 — Verify the Tool Appears in Bob	45
Exercise 8.4 — Ask Bob to Create a Service Request Using the Tool	46
Exercise 8.5 — Expose the CREATEFOLLOWUPWORKORDER Tool on MXAPIWODETAIL	48
Exercise 8.6 — Comparison of the Two Approaches	49
REST API Approach	49
Tool-Based Approach (This Lab)	49
Why the Tool Approach Is Better	49
Troubleshooting	50
Summary	50
Lab 9: Creating an Automation Script MCP Tool — Inspection Failure Advisor	51
Lab Overview	51
What You Will Learn	51
Exercise 9.1 — Create the Failure Advisor Automation Script	51
Exercise 9.3 — Configure the Automation Script as an MCP Tool	56
Exercise 9.4 — Verify Tool Availability in Bob	57
Exercise 9.5 — Use the Failure Advisor tool	57
Exercise 9.6 — Execute Additional Test Cases	59
Exercise 9.7 — Validate Natural-Language Invocation Through Bob	60
Exercise 9.8 — The Solution — LLM Semantic Classification	61
Knowledge Check	64
Troubleshooting	64
Lab Summary	65
Lab 10: Performing Analysis	66
Lab Objectives	66
Exercise 10.1 — Build the Asset Query MCP Tool Automation Script	66

Exercise 10.2 — Build the PM Query MCP Tool Automation Script	69
Exercise 10.3 — Look Up the Asset.....	72
Exercise 10.4 — Look Up the PM Records.....	73
Exercise 10.5 — Full Asset Condition and PM Analysis Report	75

Lab 1: Install Bob and Open the Workspace

This is the first lab in the series. Before you can do anything else, you need to install IBM Bob on your laptop and open the lab workspace folder. This takes about 5–10 minutes.

What You Will Do

1. Check what type of computer you have (Mac only)
2. Download and install IBM Bob
3. Obtain trial account: <https://bob.ibm.com/trial>
4. Sign in with your IBMid or Create IBMid
5. Open the lab workspace folder, provided by instructor in Bob

Before You Start

Make sure you have the following from your instructor:

What you need	Notes
IBMid email and password	Required to sign in to Bob. If you do not have one, create one by signing up for the trial.
Lab workspace folder	A folder provided by your instructor — either downloaded as a zip file or copied to a USB drive

Exercise 1.1 — Find Out What Kind of Mac You Have (Mac users only)

Bob has two Mac installers — one for newer Apple chips (M Series) and one for older Intel chips. You need to know which one to download.

1. Click the **Apple logo** () in the top-left corner of your screen
2. Click **About This Mac**
3. Look at the **Chip** line:
 - If it says **Apple M1, M2, M3** (or similar) → you need the **mac-ARM** installer
 - If it says **Intel** → you need the **mac-Intel** installer

Write down which one applies to you: _____

Windows users: Skip to Step 2 — there is only one Windows installer.

Exercise 1.2 — Download IBM Bob

Go to the Bob download page provided by your instructor and download the correct installer for your computer:

Operating System	File to download
Mac (Apple M1/M2/M3)	.pkg file — mac-ARM
Mac (Intel)	.pkg file — mac-Intel
Windows	.exe installer

Tip: If you are unsure which file to pick, ask your instructor before downloading.

Exercise 1.3 — Install Bob

On a Mac

1. Open the .pkg file you just downloaded (double-click it in your Downloads folder)
2. Follow the steps in the installation wizard — click **Continue**, **Agree**, and **Install**
3. Enter your Mac password if prompted
4. Click **Close** when the installation is complete

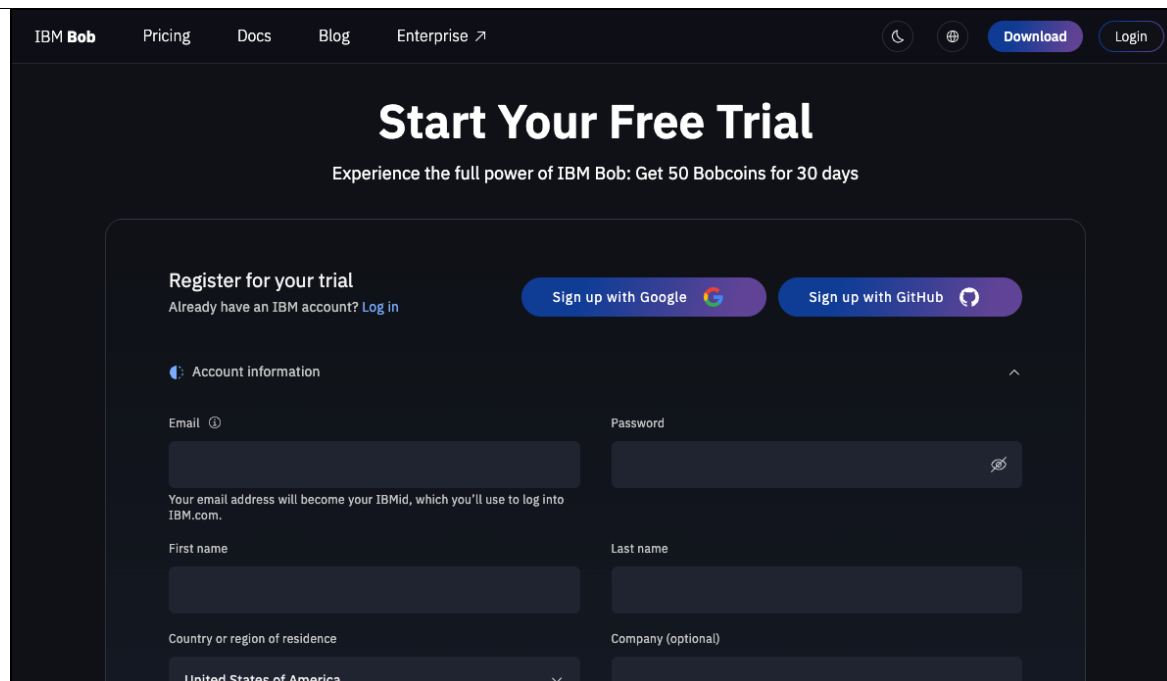
On Windows

1. Open the .exe file you just downloaded (double-click it in your Downloads folder)
2. If Windows asks “Do you want to allow this app to make changes?” click **Yes**
3. Follow the installation wizard — click **Next** and accept the defaults
4. Click **Finish** when the installation is complete

Exercise 1.4 — Sign-up for IBM Trial

The trial account provides 50 tokens or 30 days of use. **The lab will use a minimal of 30 tokens**, ensure you complete the lab before exploring other areas of the MCP service and discovering the art of the possible. NOTE: You can have multiple IBMid's.

1. Open a browser
2. Enter <https://bob.ibm.com/trial> to register for the trial environment. NOTE: You can use any accessible email address, if the email address is not associated to an IBMid and new IBMid will be created and can be used.



3. Check the email address for confirmation.
4. A second email will be sent with the IBMid if it wasn't registered and a link to login. DO NOT login using the link, this will be done on the local machine in the next exercise.

Exercise 1.5 — Launch IBM Bob

On a Mac

- Open your **Applications** folder and double-click **IBM Bob**
- Or press **⌘ Command + Space**, type Bob, and press Enter

On Windows

- Double-click the **IBM Bob** shortcut on your desktop
- Or click **Start**, search for IBM Bob, and click to open it

Bob will open and display a welcome screen or a sign-in prompt.

Exercise 1.6 — Sign In

Bob requires an IBMid to use.

1. When prompted, click **Sign in**
2. A browser window will open — enter your **IBMid email and password**
3. Complete any two-factor authentication if prompted
4. Once signed in, the browser will show a confirmation and return you to Bob automatically

If the browser does not return to Bob automatically, switch back to the Bob window manually.

Exercise 1.7 — Open the Lab Workspace Folder

The lab workspace is a folder that contains all the files Bob needs for these exercises. Your instructor will provide a link to this file.

1. Download the file to the Downloads folder and extract the file, generally right-click open with → Archive Utility or WinZip.
2. In Bob, go to **File** → **Open Folder** (Mac: top menu bar, Windows: top menu bar)
3. Navigate to the Downloads extracted downloads folder and select.
4. Click **Open**

Bob will load the workspace. You will see the folder's files listed in the left sidebar.

✓ You should see a folder called `.bob` in the file list. If you do not see it, hidden files may not be visible.

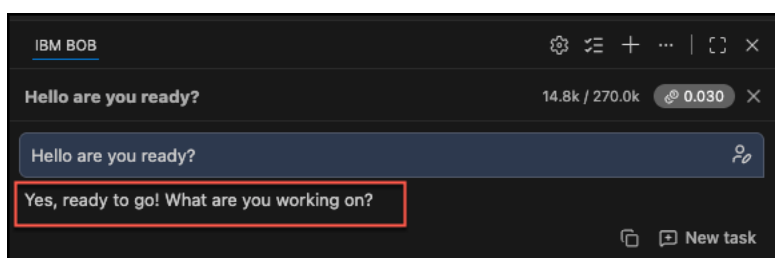
Exercise 1.8 — Verify Everything Is Working

1. In the Bob chat window at the bottom of the screen, type:

“Hello, are you ready?”



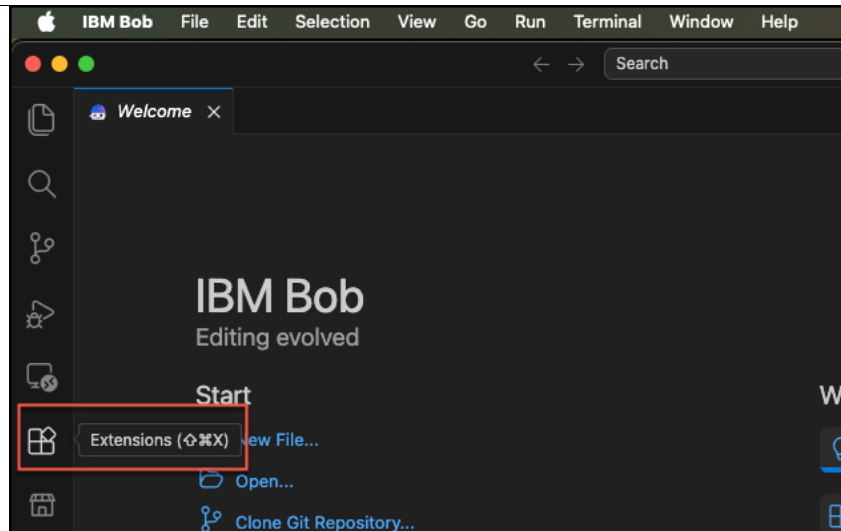
2. Bob should respond with a greeting. If it does, you are all set and ready to move on to the next lab.
NOTE: IBM Bob is NOT connected to MAS Manage at this point in the lab.



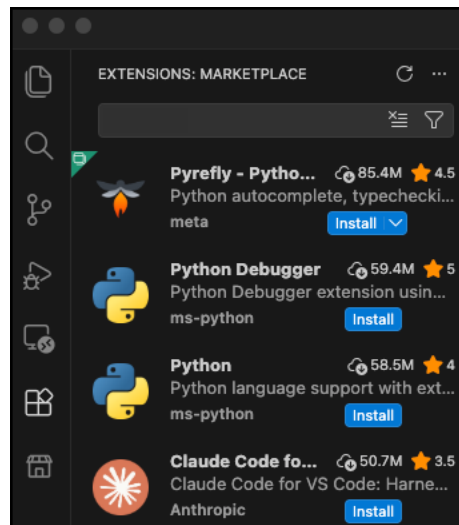
Exercise 1.9 — Install PDF Viewer

The lab guide is contained within the project folder as a pdf document. To view the guide within Bob a PDF Viewer extension must be installed.

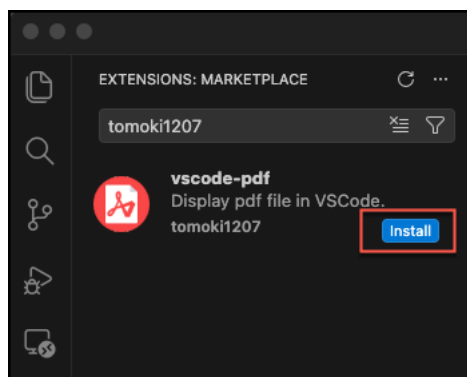
1. In the Bob IDE click the Extensions Menu icon.



- The extensions search dialog will display. A list of available extensions will display; there are numerous extensions as IBM Bob IDE is based on VSCode IDE.

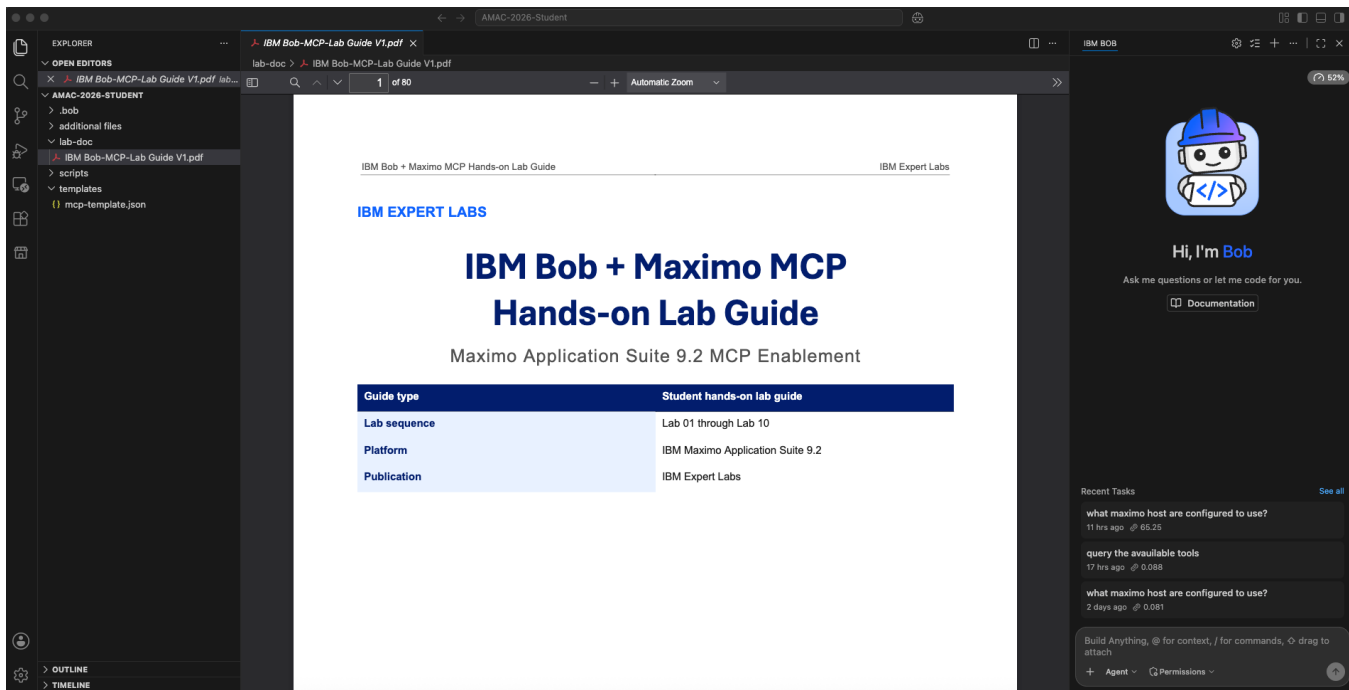


- In the search field type 'tomoki1207'. NOTE: The screenshot is illustrative and done from a system that had the extension already installed. There will be a download size 1.6M and a Star rating,



- Click Install a dialog will display asking if you trust the publisher "tomoki1207"? Click the Trust Publisher & Install button.

- Click the Explorer menu icon (A) to return to the folder and file view. Then expand the lab-doc folder and select IBM Bob-MCP-Lab Guide V1.pdf (B). This will open the lab guide inside the IDE (C) to the left of Bob Chat window.



NOTE: The lab guide may need to be updated to the more recent version, the instructor will provide that guide if that is the case.

Troubleshooting

Problem	What to do
Bob won't install on Mac	Right-click the .pkg file and choose Open — Mac security may block double-click installs for new apps
Bob won't install on Windows	Make sure you clicked Yes when Windows asked for permission
Sign-in page doesn't open	Check that your browser is not blocking pop-ups from Bob
Can't find the workspace folder	Ask your instructor — the folder must be unzipped before opening
Bob opens but shows an empty window	Use File → Open Folder to load the workspace

Summary

You have installed IBM Bob, signed in with your IBMid, and opened the lab workspace. Bob is now ready for the exercises that follow.

Next lab: *Lab 2 — Setting Up Bob to Talk to Maximo*

Lab 2: Connecting IBM Maximo to Bob via MCP

This exercise walks you through configuring a workspace-scoped MCP (Model Context Protocol) server in Bob that connects to a remote IBM Maximo Application Suite (MAS) endpoint.

Prerequisites

- IBM Bob is installed and open with this workspace folder loaded
- You have the Maximo MCP endpoint URL provided by your lab instructor
- You have a valid Maximo API key

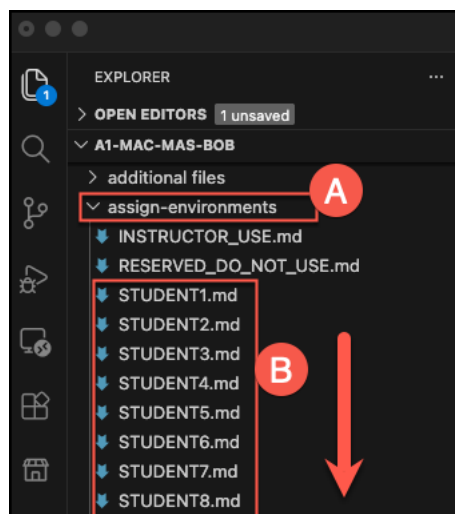
What Is MCP?

MCP (Model Context Protocol) is the mechanism Bob uses to connect to external tools and services. Each MCP server exposes a set of tools that Bob can invoke during a conversation. Configuring a **workspace-scoped** MCP server means the connection is active only within this project folder.

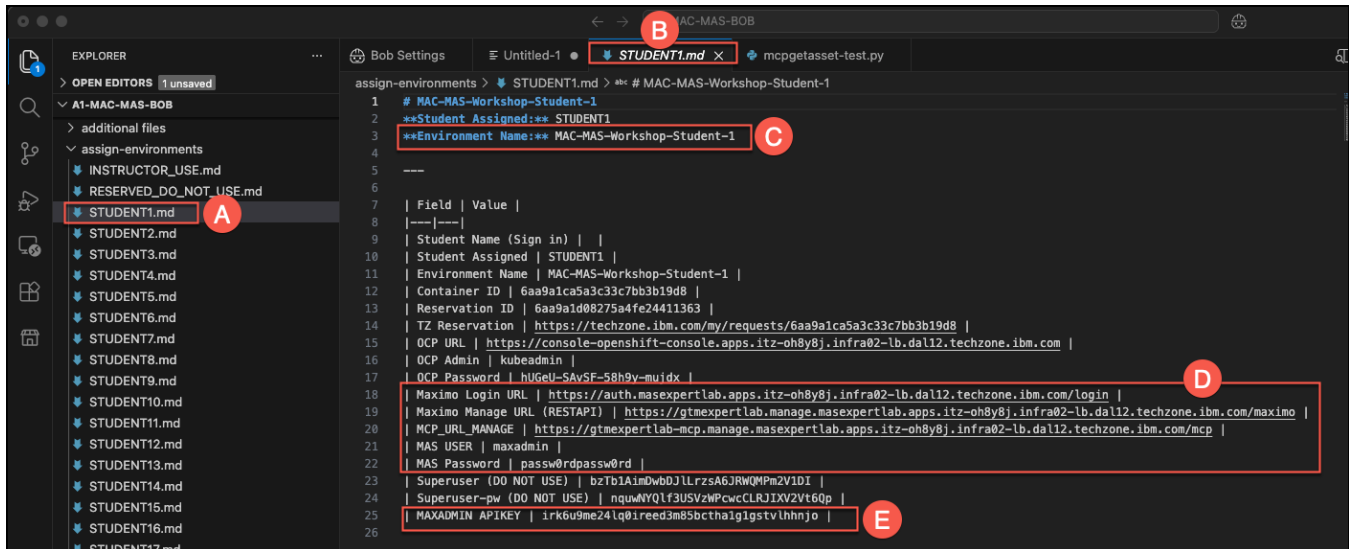
Exercise 2.1 — Obtain the MCP URL (Check Assigned-Environments folder)

When MAS Manage is installed the MCP service is installed. It runs on a pod in the Manage project and a Route is created for accessing. The route is specific to the cluster and Maximo instance. The instructor will assign the student ID which is linked to an individual environment.

1. Before proceeding **check the assigned-environments folder (A)** under the Bob project and locate your assigned environment (B) .



- Click the file for your assigned student ID (A). The file will open as a tab (B). The Environment assigned is identified in (C). The login URL's, Restapi URL and MCP URL's are identified in (D) and the MAXADMIN APIkey to use is identified in (E). These values will be used to setup IBM Bob and the MCP service.



- The remaining steps in this section are for information only and would be necessary if the information in step 2 was not provided.
- Login to OCP
- Go to Networking→Routes
- Change the Project to mas-<instance_name>-manage
- The mcp route name is the one starting with mcp, for example, mcp-masexpertlab-manage-gtmexpertlab-81.
- Under the Location column is the MCP service URL.

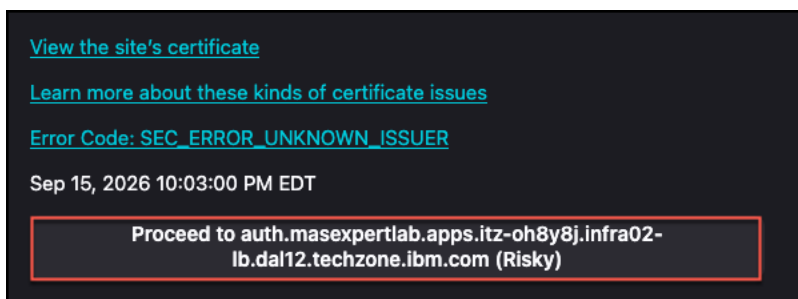
Name ↑	Status ↓	Location ↓	Service ↓
mcp-masexpertlab-manage-gtmexpertlab-81	Accepted	https://gtmexpertlab-mcp.manage.masexpertlab.apps.itz-oh8y8j.infra02-lb.dal12.techzone.ibm.com/mcp	masexpertlab-gtmexpertlab-mcp

- Copy the url and paste into a text file for use in configuring the MCP service in IBM Bob.

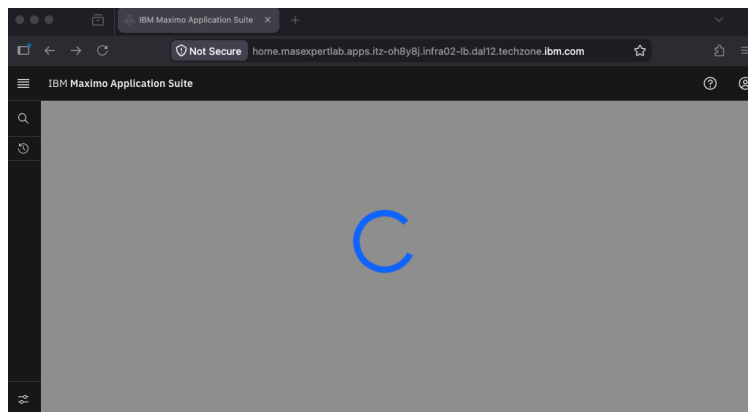
Exercise 2.2 — MAS Manage Login

The training environment uses self-signed certificates which require additional steps to login to MAS Manage the first time, after which the certificate is set. Different browsers provide somewhat different messaging to inform the user they are accessing an insecure site. Self-signed certificates should never be used in production level systems, this includes Test and Development systems supporting production.

1. Open a browser and copy and paste the Maximo Login URL from your student file.
2. The browser will display a security warning, but generally it will provide two options Advanced or Go Back. Again the wording is different per browser, moving forward is the goal.
3. Click the Advanced, again the wording varies per browser, but generally there will be a link or button that enables the user to Proceed to the website. For example, Firefox:

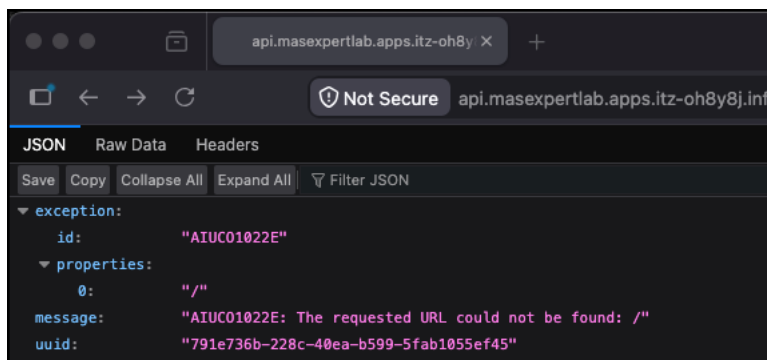


4. Click the proceed link/button.
5. You may experience another warning, clicked Advanced, then Proceed again.
6. The login page will display.
7. Login using maxadmin / passw0rdpassw0rd
8. The MAS home page will try to display, but since the self-signed certificate is not seeded the spinning cycle will just keep spinning.



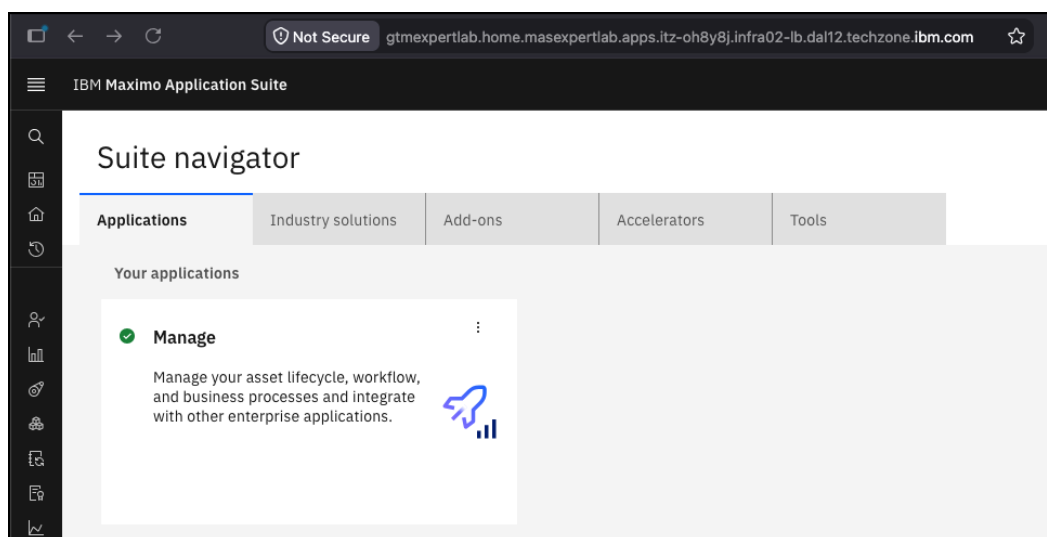
9. To seed the certificate change the 'home' value In the url <https://home.masexpertlab.apps.itz-oh8y8j.infra02-lb.dal12.techzone.ibm.com/> to <https://api.masexpertlab.apps.itz-oh8y8j.infra02-lb.dal12.techzone.ibm.com/> and press enter or refresh

10. The security warning will appear again, click advanced, then proceed. A json message will display.



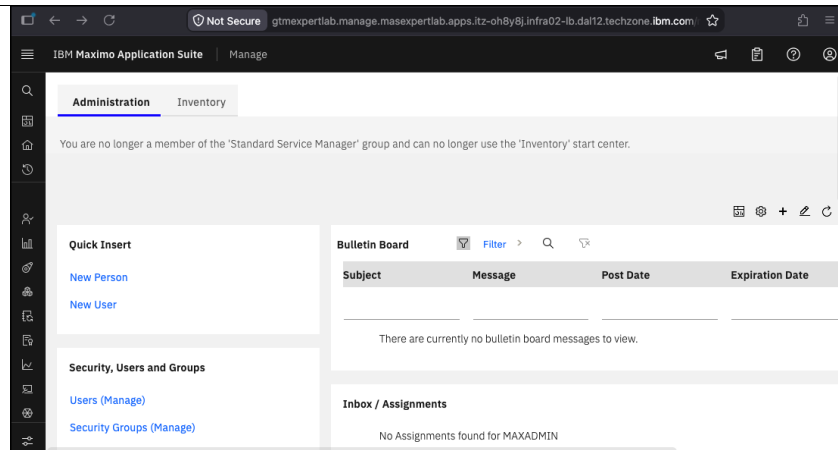
11. Click the Back button

12. Another warning message will display, click advanced and proceed and the MAS Home page will display



13. Select the Manage tile.

14. Another security warning will display, click Advanced then Proceed. The Manage start center for maxadmin will display.

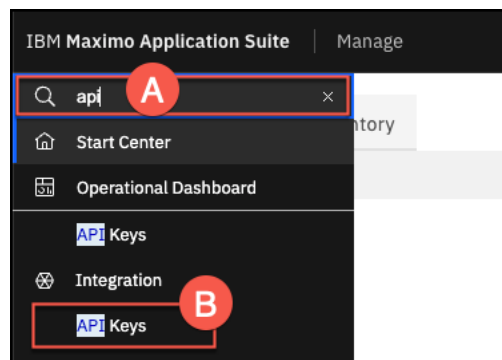


NOTE: The example used Firefox, Chrome is the browser of choice for the lab and will have less security warning.

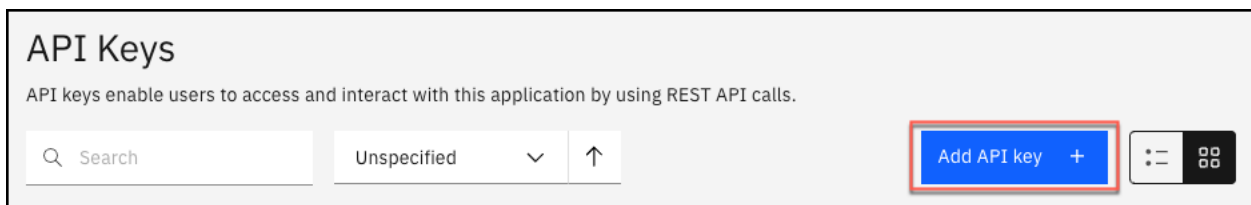
Exercise 2.3 — Create Maximo Manage API Key

IBM Bob or any other AI Agent will need to connect to Maximo Manage using an API key. Each system already has an API Key created for maxadmin, so there is no need to create a new one. However, typing the API Key is error prone and not recommended. Follow the steps to the API Key Application and copy the API Key into a test file for later use.

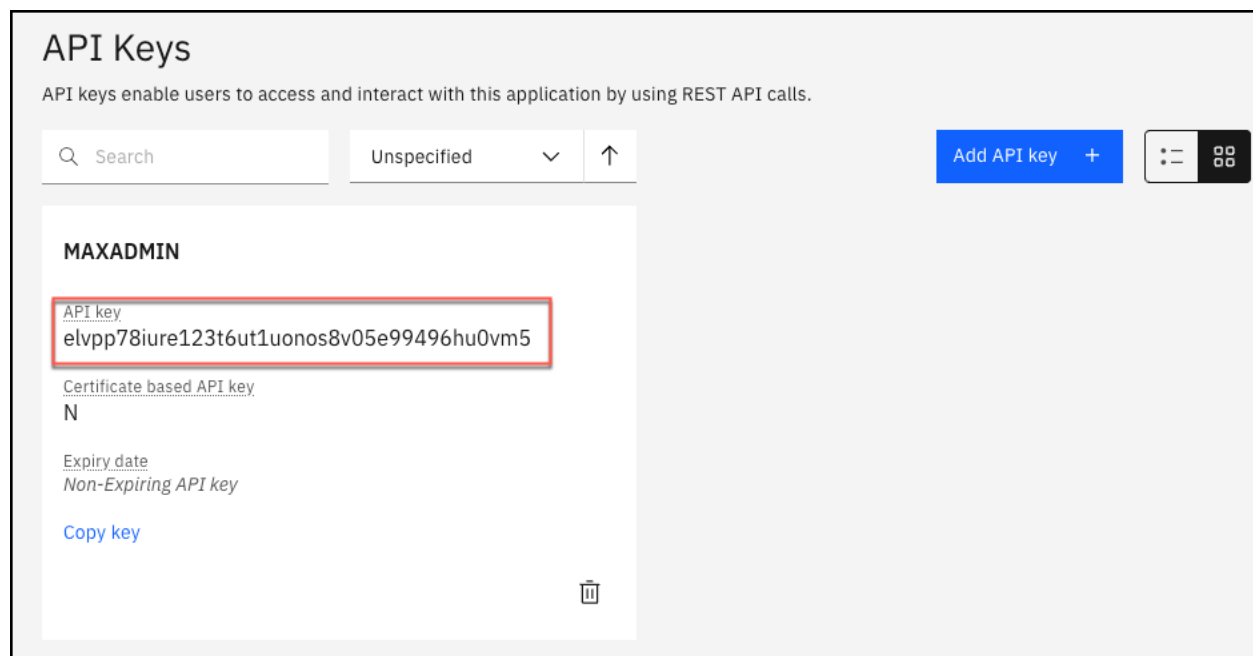
1. Use the maxadmin apikey provided from the student file. If the key does not work then proceed with the remaining steps.
2. Login to MAS Manage using the URL provided by the instructor as maxadmin using password passw0rddpassw0rd.
3. In the Find Navigation Item field enter 'api' (A), then select API Keys (B).



4. The API Keys application will launch. Click the Add API key button.

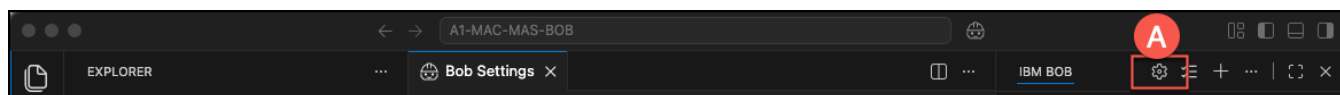


5. In the User field enter 'MAXADMIN', then click the Create button.
6. MAXADMIN API Key will get created. Copy the key and paste into a text file as it will be needed to configure the MCP Service.

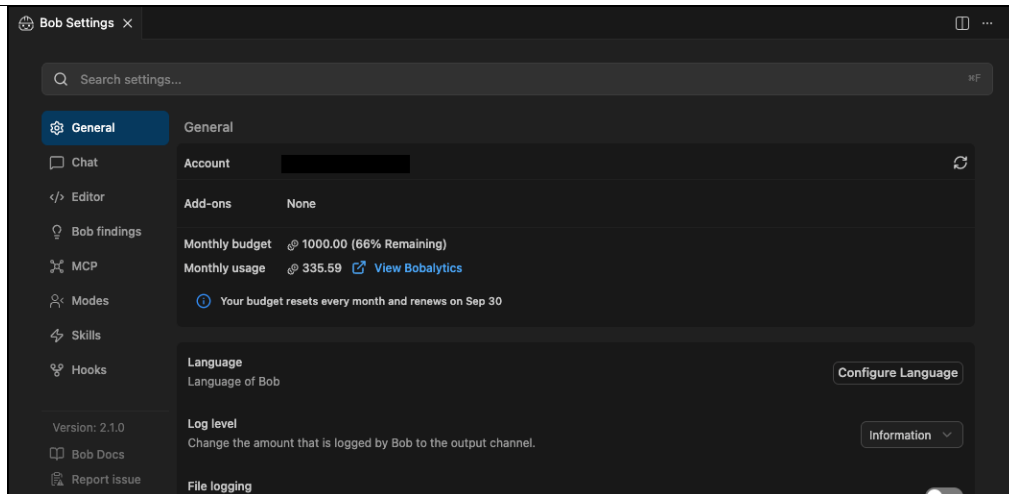


Exercise 2.4 — Locate the Workspace MCP Config File

The MCP Service is configured with IBM Bob under Setting. Accessing the menu done by clicking the Gear icon (A) located in the top right corner of Bob's UI. The lab folder provided the instructor contains a template for the MCP setting, there is no need to create a new mcp.json file.

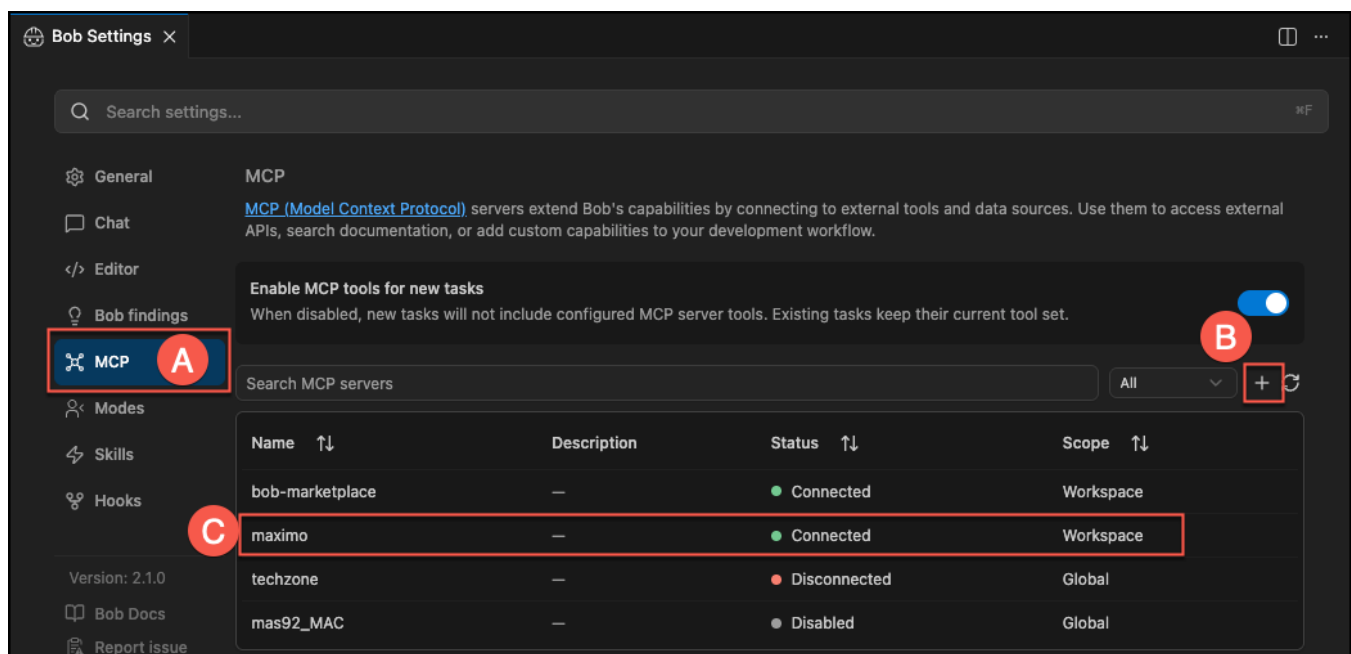


1. Click the Setting Icon (A).
2. The Setting Menu will display.



NOTE: There are many settings here, the trial account provides 50 Bob Tokens or 30 days, whichever comes first. As you progress through the lab monitor the usage, there is a cost for each transaction.

- Click the MCP Service menu (A). If no MCP Service exists, clicking the plus (+) (B) would add one. The file that is created is a json file named mcp.json. **In the lab a mcp.json template is provided and will resolve out to the student environment name assigned for the lab (C).** NOTE: IBM Bob can connect to multiple MCP instances, if working on a specific instance disable the other instances, Bob will use the cached instance so be specific as to which instance Bob needs to use if there are multiple connected instances.



- Bob reads workspace MCP configuration from `.bob/mcp.json` at the root of your workspace folder. An instance can be workspace scoped or global.

⚠ Do not overwrite existing entries — deleting another server's config will disconnect it.

Exercise 2.5 — Add the Maximo Server Entry

1. Open the `.bob/mcp.json` and add a "maximo" entry under the "mcpServers" object. This file may be hidden. Each student has an assigned environment identified by the instructor. For example, if MAC-MAS-Workshop-Student-1 was assigned, then replace the "`<replace-with-name-of-env-assigned>`" with "MAC-MAS-Workshop-Student-1".

Full file example (new file):

```
{
  "mcpServers": {
    "<replace-with-name-of-env-assigned>": {
      "type": "streamable-http",
      "url": "https://<your-maximo-mcp-host>/mcp",
      "headers": {
        "apikey": "<your-api-key>"
      },
      "disabled": false
    }
  }
}
```

2. (Reference Only) The template contains a setting for accessing the IBM Bob Marketplace, this will not be configured during the lab as is for reference only. Do not modify.

```
{
  "mcpServers": {
    "existing-server": {
      ...
    },
    "maximo": {
      "type": "streamable-http",
      "url": "https://<your-maximo-mcp-host>/mcp",
      "headers": {
        "apikey": "<your-api-key>"
      },
      "disabled": false
    }
  }
}
```

3. Replace the placeholders with the values for the mcp-url provided by your instructor and the MAS Manage API Key created in the earlier exercise:

Placeholder	Value
<your-maximo-mcp-host>	The hostname portion of the Maximo MCP endpoint URL
<your-api-key>	Your personal Maximo API key

Lab Values

Contact the instructor for the Lab URL, use the following:

Field	Value
"<replace-with-name-of-env-assigned>	Name of the environment assigned by the instructor
URL (Provided by instructor)	EXAMPLE – DO NOT USE: https://gtmexpertlab-mcp.manage.masexpertlab.apps.itz-bpufg7.infra01-lb.wdc04.techzone.ibm.com/mcp
Header name	apikey
API key	Self generated api key

Exercise 2.6 — Save the File

1. Save `.bob/mcp.json`. Bob hot-reloads MCP configuration on save — no restart is required.

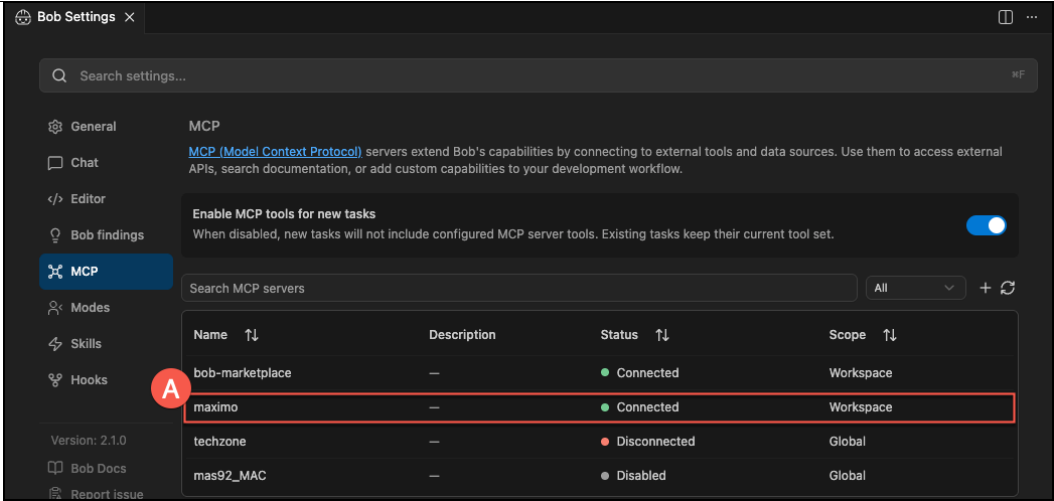
Exercise 2.7 — Verify the Connection in the Bob MCP Panel

Now check that Bob has picked up the new connection:

1. Look at the **Bob panel** on the left side of your screen
2. At the bottom of the panel, click the **Settings icon** — it looks like a gear (⚙)
3. In the settings area that opens, click the **MCP** tab
4. Look for **maximo** in the server list

What you should see next to the **maximo** entry:

- **Green dot or “Connected”** — Bob is successfully connected to the Maximo MCP service
- **Red dot, “Error”, or “Disconnected”** — something is wrong, check the items below



If the server shows an error:

- Make sure the URL is correct and matches what your instructor provided
- Make sure the API key was entered exactly as provided (no extra spaces or missing characters)
- Make sure the file was saved after editing
- Quit IBM Bob CMD+Q, and restart, the most likely issue a self-signed certificate is blocking.

If you have checked these setting and the MCP Service is still showing disconnected contact your instructor. Who will have Bob fix it.

Tip: You can click the **maximo** server entry in the MCP tab to expand it and see the individual tools it exposes. You will explore these in Lab 4.

Security Note

The API key is stored in **plaintext** inside `.bob/mcp.json` on disk. For this lab environment that is acceptable, but in production or shared repositories:

- Do **not** commit `mcp.json` to source control if it contains secrets
- Add `.bob/mcp.json` to your `.gitignore` if needed
- Prefer MCP servers that read credentials from the OS environment or a secrets manager

Reference: MCP Config Field Summary

Field	Description
type	Transport protocol — must be "streamable-http" for remote HTTP/HTTPS endpoints
url	Full URL of the remote MCP endpoint

Field	Description
headers	HTTP headers sent with every request (e.g. auth tokens)
disabled	Set to true to keep the entry but prevent connection
alwaysAllow	List of tool names that skip the confirmation prompt
timeout	Connection timeout in milliseconds
groups	Restrict this server's tools to specific Bob permission modes

Summary

You have configured a workspace-scoped MCP server named `maximo` that connects Bob to the IBM Maximo Application Suite MCP endpoint. Bob can now invoke Maximo tools directly from the chat interface during this lab session.

Next lab: *Lab 3 — Setting Up Bob to Talk to Maximo*

Lab 3: Bob Rules

Direct REST API interactions can be challenging because Bob does not always generate the optimal query on the first attempt. Instead, Bob iteratively works through the API until a valid response is returned or a blocker is encountered, such as insufficient READ access to the required object structure. This trial-and-error process can consume more tokens than expected.

To improve efficiency, the MCP Service and Maximo Tools are used whenever possible. Maximo Tools are aware of the underlying object structures, available attributes, and query templates. Query templates define both the data being requested and the expected response format, allowing Bob to retrieve information more accurately and with fewer interactions.

Tool descriptions provide another important optimization. A well-written tool description helps Bob determine when a tool should be invoked and what purpose it serves. For example, the lab includes an **Inspection Failure Advisor** tool. The tool description instructs Bob to analyze user input and map the request to one of the classification values defined within the supporting automation script. This guidance helps ensure that the correct classification is selected and that the automation script receives the expected input.

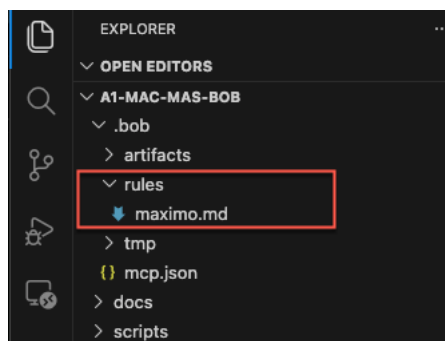
Although Bob leverages a Large Language Model (LLM) to understand requests and generate responses, questions about the specific model often result in responses such as, *"I prefer to keep that a secret,"* along with a variety of other interesting remarks. The underlying model is largely transparent to the user, as the primary focus is on the business logic, tools, and services that enable Bob to interact effectively with Maximo.

What You Will Do

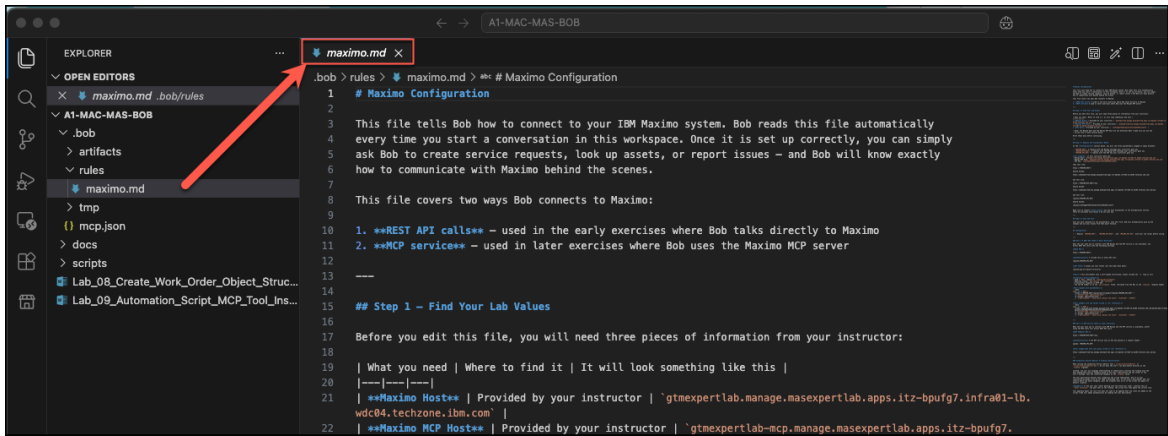
In this lab you will update the configuration section of the maximo.md file found in the rules folder in the lab files that were imported earlier.

Exercise 3.1 Update the RESTAPI and MCP Service URL's

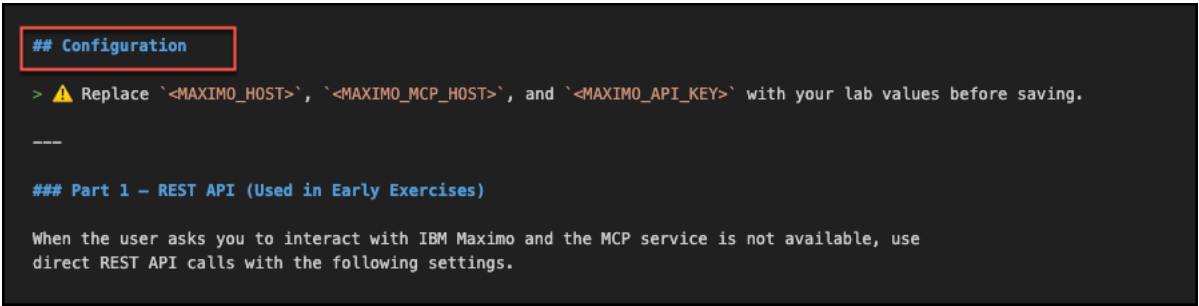
1. Expand the rules folder.



2. Select the maximo.md file. This will open the editor window.



3. Steps 1 and 2 information is provided by the instructor. Step 1 is simply the values you need, step 2 is where to place those values. Step 3 is simply telling you to save. The changes will be made in Configuration section of the file.



4. In Part 1 enter the following:

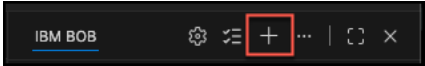
Setting	Value
Base URL	Maximo host url – this does not contain mcp in the url
apikey	The Maximo Manage apikey generated in the earlier exercise
API Path	NO CHANGE needed
SSL	NO CHANGE needed

5. In Part 2 enter the following:

Setting	Value
Base URL	Maximo MCP host url – this contains mcp in the url

apikey	The Maximo Manage apikey generated in the earlier exercise
--------	--

- 6. There is no change needed for the Inspection Failure Advisor.
- 7. There is no restart required, Bob reads this file at the start of each new chat. Since no chats have been initiate there should be no need to start a new on. However, if there one initiated click the (+) icon at the top right corner to start a new one.



- 8. There may be other entries in the file, unless other instructed please do not modify. There are there to make the best use of the trial.

Lab 4: Bob Navigation

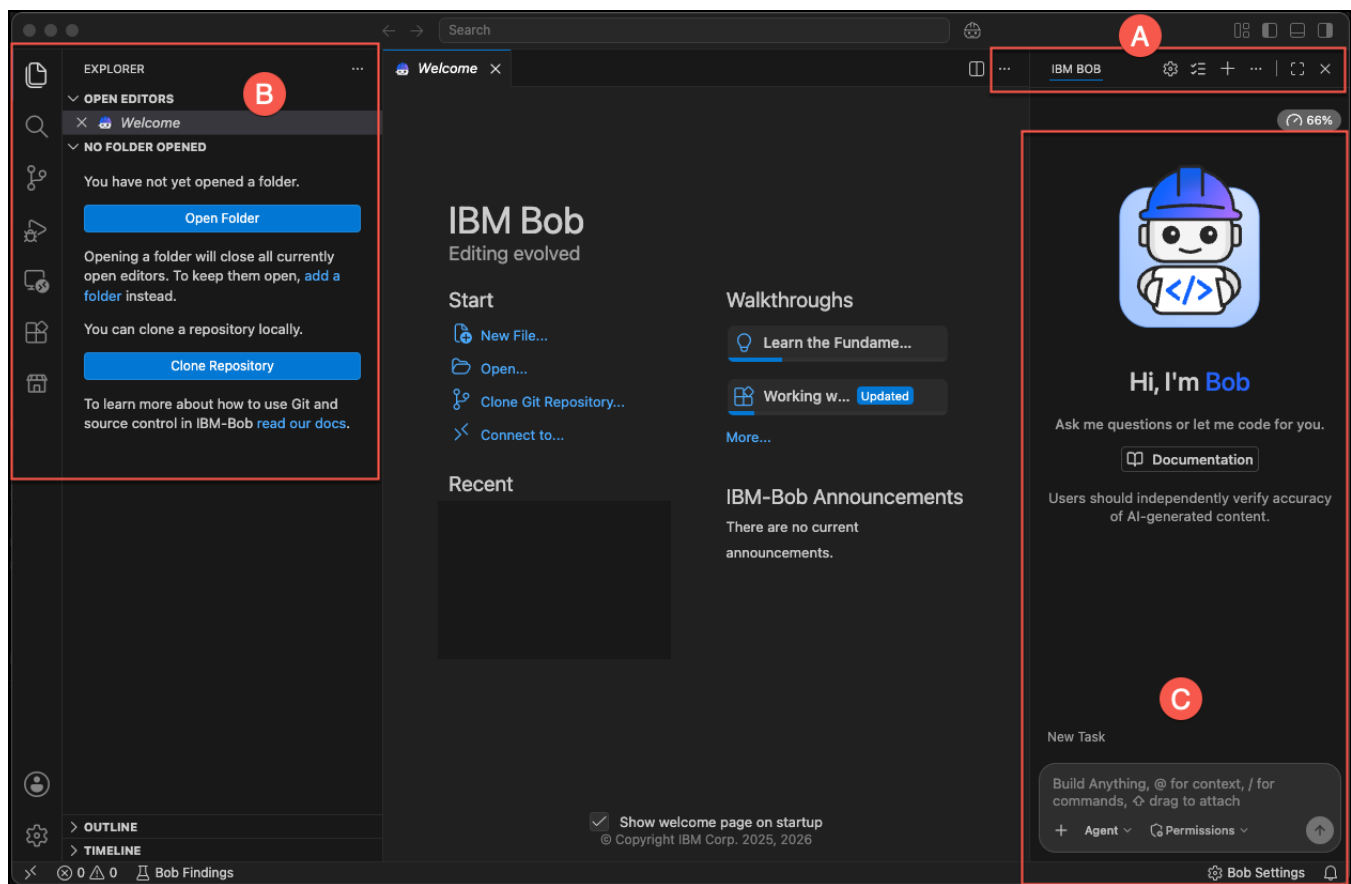
What You Will Do

This lab covers the basic navigation in IBM Bob. IBM Bob's UI is VSCode so those familiar with that application will be very comfortable. In this exercise you will create a configuration file that tells Bob how to connect to your IBM Maximo system. You only need to do this once. Once the file is in place, Bob will automatically know how to communicate with Maximo every time you start a conversation in this workspace.

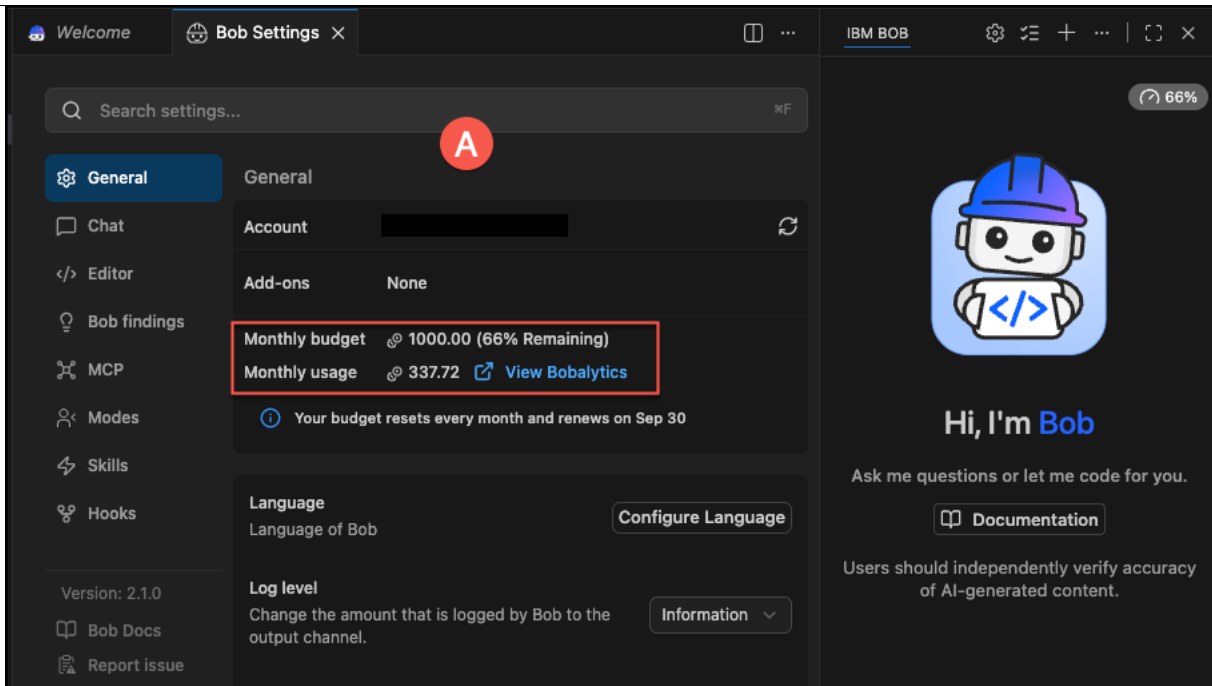
You do not need any technical knowledge to complete this exercise — just follow the steps below.

Exercise 4.1 - Navigation

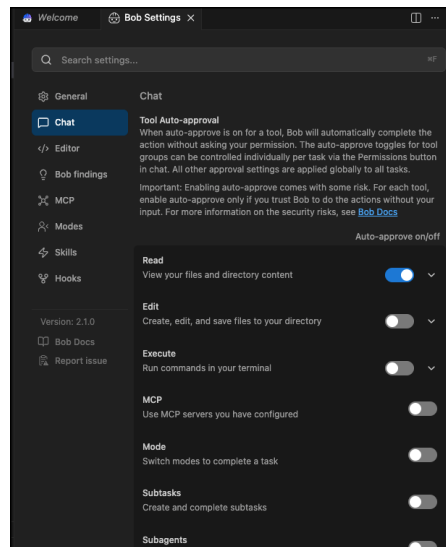
1. The top right menu (A) has been discussed already the gear icon provides access to the setting, chat history and the ability to start a new task (chat). In the explorer panel (B) will display the project folder. The IBM Bob chat UI (C) is used to communicate with Bob.



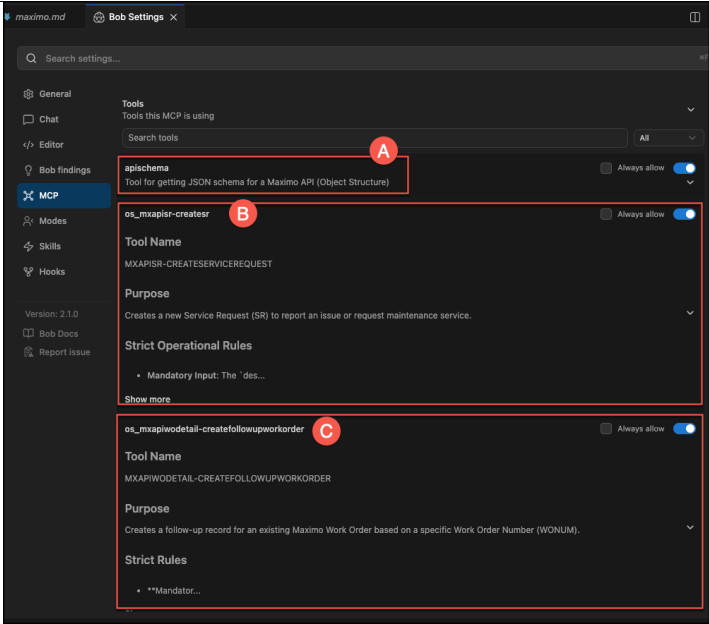
2. Click the Gear icon to open Bob's Setting (A).
3. On the Setting General Tab (A) is the Monthly Budget, which is generally reset at the end each month.



4. The Chat tab controls features such as auto-approval which allows Bob to accomplish those tasks without seeking your approval. As the important note states be careful with these features, it can be a little annoying to approve everything but then again having to undo a mistake because Bob got the context wrong or maybe the user did or it introduces a security issue.



5. The other important one for this workshop is the MCP service, which was discussed in the MCP Setup. The initial setup and restart of the mcp is just one tasks, this tab also displays the Tools that are available to the service. At this point in the lab only the maximo apischema (A) is available, which helps Bob determine which objects and attributes it needs to build rest calls to Maximo. As the lab builds additional tools will become available for example, os_mxapisr-createsr (B) is an Object Structure tool, as is os_mxapiwodetail-createfollowupworkorder (C). Later in the lab this list will grow as automation script as a tool are deployed.



6. IBM Bob comes with specific Modes and Skills, which can be extended. Modes are specializing personas that tailor Bob's behavior. For example, there is a Plan mode that analyzes requirements, research and designs implementation steps. There is the Agent mode, which brings your idea or plan to life. An Ask mode which is used, yes to ask questions and get answers. You can add enable additional modes from Bob's marketplace or create modes. Along with Modes there are skills. Skills are reusable instruction sets, a recipe, that teach Bob new a talent. When you activate a skill, Bob receives the skill's instructions and gains access to any supporting files in the skill directory. Bob then follows these instructions to complete your task according to the defined workflow.

General

Chat

Editor

Bob findings

MCP

Modes

Skills

Hooks

Version: 2.1.0

Bob Docs

Report issue

Modes

Modes are specialized personas that tailor Bob's behavior. Learn about [Using Modes](#) or [Customizing Modes](#).

Search Modes

All

Name	Description	Scope
Agent	Take your idea, or plan, and bring...	Built-in
Plan	Plans tasks: analyzes requiremen...	Built-in
Ask	Ask questions and get explanatio...	Built-in
Documentation Writer	Produce high-quality documentat...	Global
User story Builder	Generate actionable and well-def...	Global
Carbon React	(2.5.0) Create Carbon React com...	Global
Simplify	Simplifies and refines code AND ...	Global

Rows per page: 20Page 1 of 1

General

Chat

Editor

Bob findings

MCP

Modes

Skills

Hooks

Version: 2.1.0

Bob Docs

Report issue

Skills

Skills are reusable instruction sets that teach Bob new workflows and specialized tasks. Think of them as recipes that Bob follows to complete specific types of work in a consistent, repeatable manner. [Learn about Skills](#)

Search skills...

All

Name	Description	Scope	Active
create-plan	Structured workflow for gatherin...	Built-in	Always on
create-skill	Guide for creating a new Bob skill...	Built-in	Always on
create-mode	Guide for creating a new custom ...	Built-in	Always on
configure-hooks	Add, update, remove, or troubles...	Built-in	Always on
configure-mcp	Add a new MCP server or diagno...	Built-in	Always on
build-mcp-server	Guide for building a custom MCP ...	Built-in	Always on
office-insights	Use when the user wants to read...	Built-in	Always on
create-chart	Full component schemas for the ...	Built-in	Always on

Rows per page: 100Page 1 of 1

Next lab: Lab 5 — Discover What Maximo Tools Are Available

Lab 5: Discover What Maximo Tools Are Available

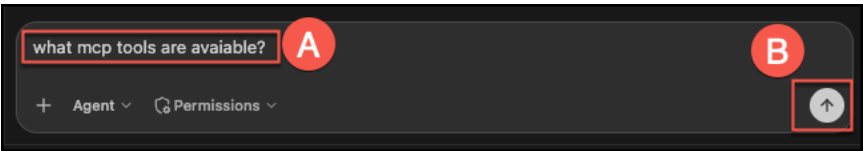
In this exercise you will ask Bob what Maximo tools are available and then try one of them. This is a short exercise — you will simply type a couple of messages in the Bob chat window and review what comes back.

What You Will Do

- 1. Ask Bob to list the available Maximo MCP tools
- 2. Review what Bob finds
- 3. Use the available tool to pull live information from Maximo

Exercise 5.1 — Ask Bob What Tools Are Available

- 1. In the Bob chat window, type the following and click send message:
“What Maximo MCP tools are available?”



- 2. Bob will check the Maximo MCP connection and tell you what tools it has access to. At this stage of the lab, you should only see one tool.

Based on the MCP tools currently registered in this session, here's what's available from the Maximo server:

Maximo MCP Tools		
Tool	Type	Description
apischema	Introspection	Get JSON schema for a Maximo API (Object Structure)

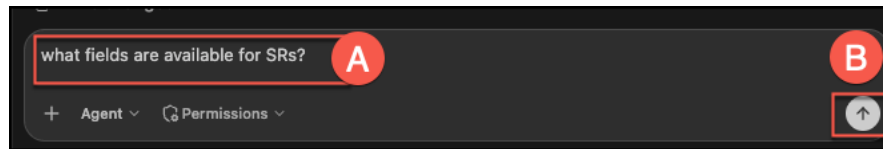
Tool Name	What It Does
apischema	Looks up the fields and data types available on any Maximo record type. For example, it can tell you what fields exist on a Service Request or a Work Order.

Note: If Bob lists more tools than just `apischema`, additional tools have been added to the Maximo MCP service since this lab was written — that is a good thing! Make a note of what you see.

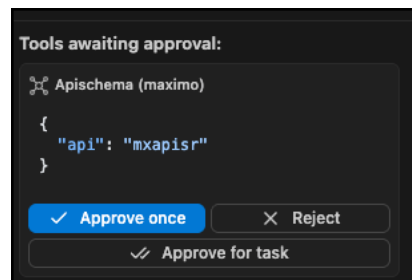
Exercise 5.2 — Use the Tool

1. Ask Bob to use the `apischema` tool. Type the following in the chat dialog (A) and click the **Send Message button (B)**:

“What fields are available for SR’s?”



2. Bob will call the tool, determine the schema to use, then ask for Approval to execute.



3. Approve the request and Bob will return the key fields sorted by categories. You do not need to understand every field — the point is to see Bob reaching out to your live Maximo system and returning real data based on a plain language request.

Relationships	
Field	Description
<code>origrecordid</code>	Originating record (SR/WO that spawned this)
<code>origrecordclass</code>	Class of originating record
<code>globalticketid</code>	Linked global ticket
<code>solution</code>	Applied solution ID
Flags	
Field	Description
<code>hassafetyhazard</code>	Critical safety incident flag
<code>isglobal</code>	Global issue flag
<code>sitevisit</code>	On-site visit required
<code>inheritstatus</code>	Inherit status from follow-up record
For creating a new SR via the MCP tool, the most commonly used fields are <code>description</code> , <code>assetnum</code> , <code>location</code> , <code>siteid</code> , <code>reportedby</code> , and <code>reportedpriority</code> .	

What This Means

The `apischema` tool is a **discovery tool** — it tells Bob what information Maximo holds for any type of record. Bob uses this behind the scenes to understand Maximo’s structure.

Right now this is the only tool available on this MCP instance. As more tools are added, Bob will be able to do more — such as creating service requests, moving assets, or reporting downtime — directly through the MCP connection, without having to build restapi calls as the tool tells Bob what is available.

Why Only One Tool?

MCP tools are added by the team that manages your Maximo environment. Think of it like an app store — the store is open, but only certain apps have been published so far. The `apischema` tool is the first one available.

In the next exercise you will see how Bob works around this by using a direct connection to Maximo to create a service request, even before the full set of MCP tools is in place.

Troubleshooting

Problem	What to do
Bob says it cannot find any tools	Go back to Lab 2 and confirm the maximo server shows as connected in the MCP panel
Bob says it cannot connect	Make sure Lab 3 was completed and the rule file was saved with the correct MCP host value
Bob asks me to approve something	Click Allow — Bob is asking permission before calling the tool
Bob returns a different list of tools	More tools may have been added — note them down and tell your instructor

Summary

In this exercise you:

1. Asked Bob what Maximo MCP tools are available
2. Learned about the `apischema` tool currently available on this instance
3. Used the tool to retrieve live field information from your Maximo system

Next lab: *Lab 6 — Ask Bob to Create a Maximo Service Request*

Lab 6: Ask Bob to Create a Maximo Service Request

In this exercise, Bob will be used to report a maintenance issue in IBM Maximo. No technical knowledge is required. Simply describe the problem in plain language, just as a message would be sent to a colleague.

NOTE 1: Bob works like a colleague assisting with a task. Sometimes a request may be misunderstood, a detail may be omitted, or additional information may be needed to complete the task. When this occurs, clarify the request and provide the missing information.

NOTE 2: Rules have been created to reduce make the best use of the trial. Bob will confirm your intent before executing for this lab. To turn them off simply tell Bob to stop confirming intent, Bob will remember until you start a new chat.

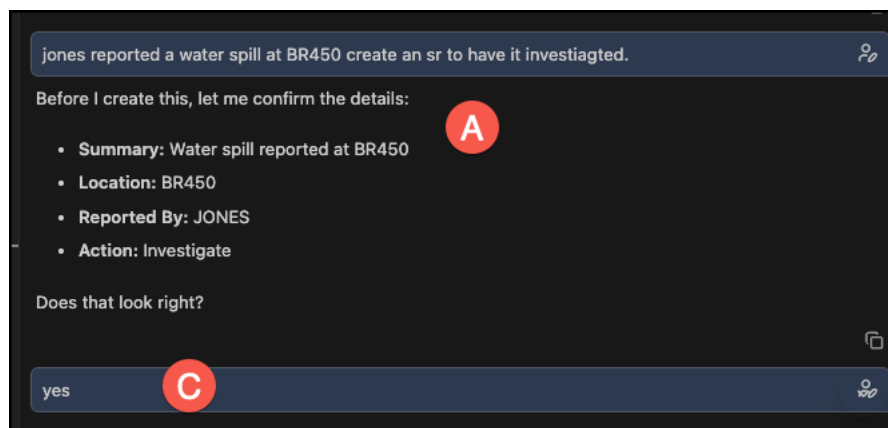
Exercise 6.1 — Report the Issue

In this exercise employee Jones reports a loud noise coming from the motor of asset 11450.

1. Type the following message in the chat window of Bob and press **Send Message**:

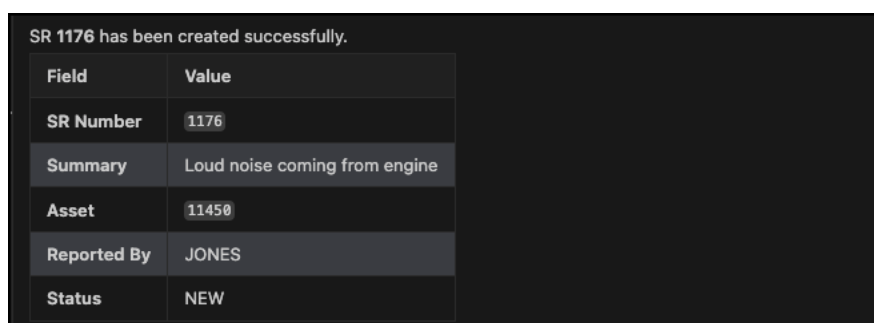
“Jones reported a loud noise coming from the motor of 11450, create an sr”

2. Bob will confirm the intend (A), then confirm (B).



The screenshot shows a chat interface. At the top, a message from 'jones' says: 'jones reported a water spill at BR450 create an sr to have it investiagted.' Below this, a confirmation message from Bob says: 'Before I create this, let me confirm the details:'. This is followed by a list of details: 'Summary: Water spill reported at BR450', 'Location: BR450', 'Reported By: JONES', and 'Action: Investigate'. Below the list, it asks 'Does that look right?'. At the bottom, there is a response box with the word 'yes' and a red circle with the letter 'C' next to it, indicating a confirmation action.

3. After a few seconds, Bob will reply with a ticket information. It will look something like this:



The screenshot shows a table with the following data:

SR 1176 has been created successfully.	
Field	Value
SR Number	1176
Summary	Loud noise coming from engine
Asset	11450
Reported By	JONES
Status	NEW

4. **Write down your SR Number** — you will need it in the next exercise. _____

Exercise 6.2 — Try Your Own Request

1. Now try asking Bob about a different issue. You can use your own words this time. For example:
“Log a service request — the lights are flickering in room 3 near asset 11450” or “There is a water leak near asset 11450, please raise a service request”
2. Notice how Bob understands what you mean even when you phrase it differently. Since the reported by field was used Bob may prompt for the reported by user.

What Just Happened?

When you typed your message, Bob:

1. Understood that you wanted to report a maintenance issue
2. Identified the asset number (**11450**) from your message
3. Used the Maximo connection you configured in the previous lab to log the request
4. Confirmed the new **Service Request number** so you can track it

You did not need to log into Maximo, fill out a form, or know anything about the system. Bob handled all of that for you.

If Something Goes Wrong

What you see	What to do
Bob says it cannot connect	Make sure you completed the rule setup lab and saved the file correctly
Bob asks for more information	Provide the asset number (11450) and a short description of the problem
Bob gives an error message	Copy the message and show it to your instructor

Summary

In this exercise you reported a maintenance issue to Maximo simply by describing it to Bob. No forms, no logins, no technical steps — just a plain language message.

This is the power of AI-assisted operations: Bob acts as your bridge to enterprise systems, so you can focus on describing the problem rather than navigating software.

Lab 7: Talking to Maximo in Plain Language

In the previous labs, the connection between IBM Bob and Maximo was established. This lab puts that connection to work.

The focus is not on technology. There is no need to write code, build API calls, or search documentation. Simply describe the desired outcome in plain language, the same way a request would be made to a knowledgeable colleague, and IBM Bob handles the rest.

Behind the scenes, IBM Bob constructs the required REST API calls, manages authentication, resolves record relationships, encodes identifiers, follows linked records, and handles error responses automatically. Tasks that might take a developer significant time to build, test, and troubleshoot can be completed in seconds. The only requirement is knowing what information is needed or what action is being requested.

The trial environment includes **50 tokens** to experiment with IBM Bob. The exercises in this lab typically use about **28 tokens**, leaving approximately **22 tokens** to explore your own questions, scenarios, and ideas. Be conservative until you complete the lab.

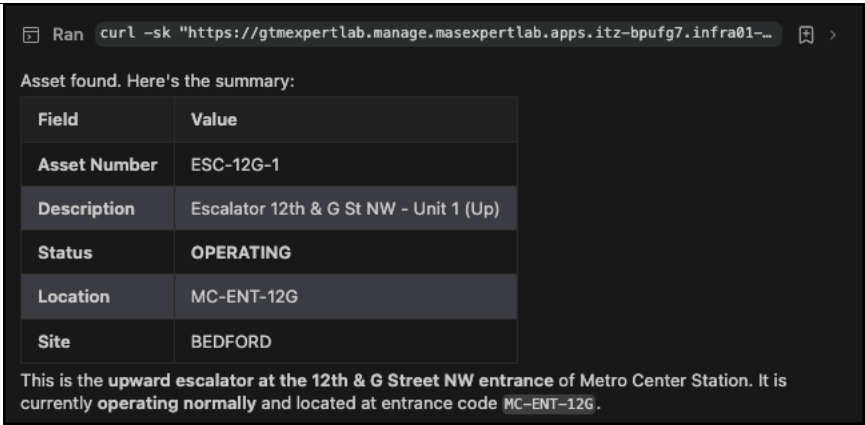
Make the most of them and see what IBM Bob can accomplish through simple natural language conversations with Maximo.

What You Will Do

Work through the exercises below in order. Each one builds on the last. Type each prompt into the Bob chat window and review what Bob returns before moving on.

Exercise 7.1 — Look Up an Asset

1. Type the following into the Bob chat window and press **Send Message**:
“Tell me about asset ESC-12G-1 at Metro Center Station. What is its current status, location, and description?”
2. Bob will retrieve the asset record from Maximo and summarize it for you. Notice how you asked a question the same way you would ask a person — Bob translated that into a live lookup against your Maximo system.



NOTE: If the query fails add in a phase to use the BEDFORD site.

Exercise 7.2 — Explore a Station’s Assets

- Now ask Bob for a broader view:
“List all escalator and elevator assets at Metro Center Station. Show the asset number, description, location, and current status.”
- Bob will query Maximo and return the full list. With 18 assets across multiple entrances and levels, this would take considerable time to gather manually. Bob returns it in seconds. NOTE: Bob may execute more than one query.

Here's the full picture. 18 assets found — 12 escalators and 6 elevators across Metro Center Station.			
Station Status Summary — 3 of 18 OPERATING, 15 NOT READY			
Escalators (12)			
Asset	Description	Location	Status
ESC-12G-1	Escalator 12th & G St NW — Unit 1 (Up)	MC-ENT-12G	OPERATING
ESC-12G-2	Escalator 12th & G St NW — Unit 2 (Down)	MC-ENT-12G	OPERATING
ESC-12F-1	Escalator 12th & F St NW — Unit 1 (Up)	MC-ENT-12F	NOT READY
ESC-12F-2	Escalator 12th & F St NW — Unit 2 (Down)	MC-ENT-12F	NOT READY
ESC-13G-1	Escalator 13th & G St NW — Unit 1 (Up)	MC-ENT-13G	NOT READY
ESC-13G-2	Escalator 13th & G St NW — Unit 2 (Down)	MC-ENT-13G	NOT READY
ESC-11G-1	Escalator 11th & G St NW — Unit 1 (Up)	MC-ENT-11G	NOT READY
ESC-11G-2	Escalator 11th & G St NW — Unit 2 (Down)	MC-ENT-11G	NOT READY
ESC-UM-RL-1	Escalator Upper Mezz to Red Line — Unit 1	MC-UMEZZ	NOT READY
ESC-UM-RL-2	Escalator Upper Mezz to Red Line — Unit 2	MC-UMEZZ	NOT READY
ESC-UM-LM-1	Escalator Upper Mezz to Lower Mezz — Unit 1	MC-UMEZZ	NOT READY
ESC-LM-LL-1	Escalator Lower Mezz to Lower Platform — Unit 1	MC-LMEZZ	NOT READY

Exercise 7.3 — Filter the Results

1. You do not need to re-run the query to narrow it down. Bob remembers the context of your conversation. Try:

“Now just show me the assets at 12th and G.”

2. Bob will confirm your intent:

“What would you like to do with the assets at 12th & G once you have the list?”

3. Bob will filter the previous results without going back to Maximo. Notice that you did not need to repeat the asset type, the station name, or any other detail — Bob understood what “now just show me” meant in context.

Asset	Description	Status
ESC-12G-1	Escalator 12th & G St NW — Unit 1 (Up)	
ESC-12G-2	Escalator 12th & G St NW — Unit 2 (Down)	
ELV-12G	Elevator 12th & G St NW	

All 3 assets at this entrance are fully operational.

Exercise 7.4 — Update an Asset Status

1. Ask Bob to make a change:

“Change their status to OPERATING for these assets.”

2. Bob will identify the three assets from your previous conversation and ask you to confirm.

To confirm — I'll update all 3 assets at 12th & G to Operating status

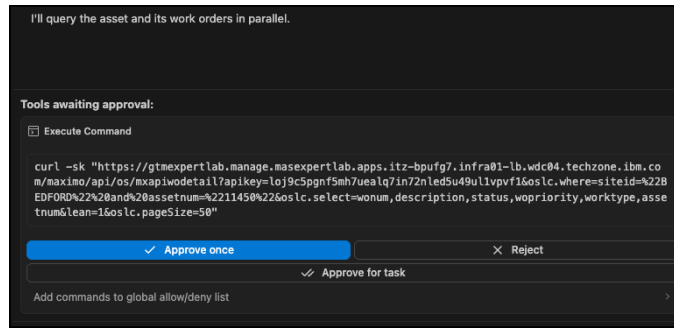
3. Type Yes to proceed.
4. Bob may ask permission to check the Asset Status, grant permission. I.e. Approve and check the I understand the risk checkbox. Bob will look up their internal Maximo record identifiers and update each one. Status changes in Maximo normally require navigating to each record individually, finding the status field, and saving. Bob handles all three in one instruction.

Assets at 12th & G St NW Entrance (MC-ENT-12G)		
Asset	Description	Status
ESC-12G-1	Escalator 12th & G St NW — Unit 1 (Up)	✓ OPERATING
ESC-12G-2	Escalator 12th & G St NW — Unit 2 (Down)	✓ OPERATING
ELV-12G	Elevator 12th & G St NW	✓ OPERATING

All 3 assets at this entrance are fully operational.

Exercise 7.5 — Look Up Work Orders for an Asset

1. Ask Bob to:
“Show me asset 11450. What work orders exist for it? Give me the work order number, description, status, and priority.”
2. Notice that asked for two things, Asset and Work orders, Bob handled both together.



NOTE: There will be two queries that run.

3. Bob will return the full work order history for the asset.

62 work orders total. Here's the full organised summary:

Asset 11450 — Work Orders (62 total)

● WAPPR — Waiting Approval (26)

WO Number	Description	Priority
1201	Centrifugal Pump Service - 3 Month	8
1209	Centrifugal Pump Service - 3 Month	8
1210	Centrifugal Pump Service - 3 Month	8
T1074–T1079	PM tasks: check operation, float switch, leaks, impeller, electrical	8
T1080–T1085	PM tasks (second set)	8
T1086–T1091	PM tasks (third set)	8
1202–1204	Noise coming from engine	—
1207	Loud noise coming from engine	—
1208	Failed motor	1
1211	Reinspection after seal replacement	3

Exercise 7.6 — Find a Job Plan

1. Ask Bob to find Job Plans for monthly pump maintenance:
“What job plan exists for monthly pump maintenance? Show me the tasks and materials.”

- Bob will search Maximo for relevant job plans and return their task lists and material requirements. You described what you were looking for in plain terms — Bob worked out which records to query and how to join the data together. It took 7 queries to get the results.

```
Ran 2 commands >

MXAPIJP isn't exposed in this environment. Let me try the standard job plan object structure names.

Ran curl -sk "https://gtmexpertlab.manage.masexpertlab.apps.itz-bpufg7.infra01-lb.wdc04.techzone.i... >

Two pump job plans found — JP11430 (12 month) and JP11430A (3 month). Let me pull both with their tasks and materials.

Ran 5 commands >

Ran curl -sk "https://gtmexpertlab.manage.masexpertlab.apps.itz-bpufg7.infra01-lb.wdc04.techzone.i... >
Ran curl -sk "https://gtmexpertlab.manage.masexpertlab.apps.itz-bpufg7.infra01-lb.wdc04.techzone.i... >
Ran curl -sk "https://gtmexpertlab.manage.masexpertlab.apps.itz-bpufg7.infra01-lb.wdc04.techzone.i... >
Ran curl -sk "https://gtmexpertlab.manage.masexpertlab.apps.itz-bpufg7.infra01-lb.wdc04.techzone.i... >
Ran curl -sk "https://gtmexpertlab.manage.masexpertlab.apps.itz-bpufg7.infra01-lb.wdc04.techzone.i... >
```

- Bob returned one Job Plan, identified missing parts and the work it covers, and found a 12-month version that can also be viewed.

Here's the job plan for the 3-month pump service:

Job Plan: JP11430A — Centrifugal Pump Service (3-Month)

Status: ACTIVE | Site: BEDFORD

Tasks

Task	Description	Duration
10	Check pump operation	0.17 hrs (~10 min)
20	Check pump float switch	0.50 hrs (~30 min)
30	Check seal and housing for leaks	0.17 hrs (~10 min)
40	Inspect impeller for wear	0.17 hrs (~10 min)
50	Record amperage and voltage with motor operating	0.25 hrs (~15 min)
60	Inspect main disconnect and all electrical	0.25 hrs (~15 min)

Total estimated duration: ~1.5 hrs

Materials

No materials are listed against this job plan.

Notable: This is the job plan referenced by the WAPPR work orders 1201, 1209, and 1210 on asset 11450 (the tasks T1074–T1091 are the spawned child tasks from those WOs). The plan covers operational checks, leak inspection, impeller wear, and electrical — but has no materials, meaning consumables like seals or gaskets would need to be added at the work order level.

A 12-month version (JP11430) also exists in Maximo if you'd like to see that one too.

Exercise 7.7 — Create a Work Order from a Job Plan and Link It to an SR

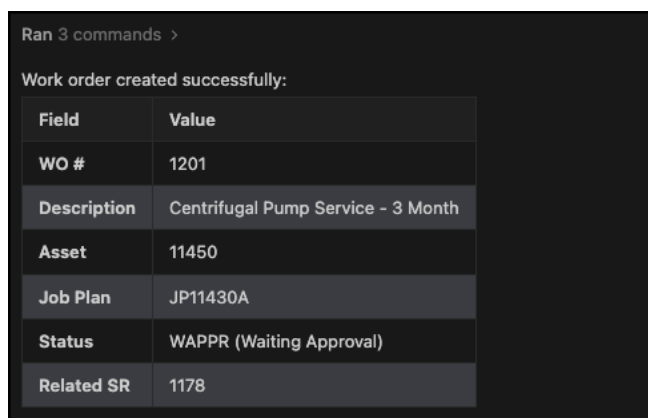
1. For this exercise you will need the SR number you created in **Lab 6**. Check your notes from that lab now. If you have your SR number, type the following — replacing <your SR number> with the number you wrote down:

“Use JP11430A and create a work order for 11450. Associate the work order to SR <your SR number>.”

2. (Optional) If you cannot remember your SR number, ask Bob, i.e., what was the last SR I created. Or type this instead. You can always link the SR later once you look it up.

“Use JP11430A and create a work order for asset 11450.”

3. Bob may confirm your intent, type ‘Yes’ to proceed.
4. Bob will:
 - Create a new work order against asset 11450 using job plan JP11430A
 - Set the originating record to your SR (if you provided the number)
 - Return the new work order number and confirm the details



Ran 3 commands >

Work order created successfully:

Field	Value
WO #	1201
Description	Centrifugal Pump Service - 3 Month
Asset	11450
Job Plan	JP11430A
Status	WAPPR (Waiting Approval)
Related SR	1178

5. Write down your new **work order number** — you may need it in a later exercise.

NOTE: This single instruction replaced what would otherwise require: opening a work order form, filling in asset details, selecting the job plan, navigating to the related records tab, clicking the add new row, searching for the SR and saving.

What Just Happened?

Across these seven exercises you:

- Retrieved and filtered asset records
- Updated asset statuses
- Reviewed work order history
- Discovered job plans with tasks and materials

- Created a work order linked to a service request

You did all of this without logging into Maximo, without knowing any API endpoints, without building any queries, and without understanding how record identifiers or authentication headers work. You described your intent and Bob handled the rest.

Key Takeaway

The value Bob brings is not just automation — it is **removing the barrier between what you know and what the system needs**. You know the problem: a noisy pump, an out-of-service escalator, a maintenance task that needs scheduling. Bob knows how to translate that into action inside Maximo.

Together, you can move from problem to resolution faster than any traditional workflow allows.

Troubleshooting

Problem	What to do
Bob says it cannot find the asset	Check the asset number — try asking “What assets exist at Metro Center?” first
Bob returns an error on status update	The asset may already be at that status, or your API key may not have update permissions — ask your instructor
Bob creates the work order but can’t link the SR	The SR number may be wrong — confirm it from Lab 5 before retrying
Bob asks a clarifying question	Answer it — Bob may need one more detail before it can act
Bob says that siteid is required	In the prompt add test such as the asset is in the Bedford site

Summary

In this lab you experienced what it means to use natural language as your interface to an enterprise system. Every interaction was a plain English instruction. Every result came directly from your live Maximo environment.

This is what AI-assisted operations looks like in practice.

Lab 8: Enabling Maximo Object Structure Tools for Bob

In the previous labs Bob created service requests and work orders by building REST API calls behind the scenes. In this lab you will enable a native Maximo tool that lets Bob interact with Maximo more directly — using an **Object Structure Tool** that Maximo publishes specifically for AI assistants like Bob.

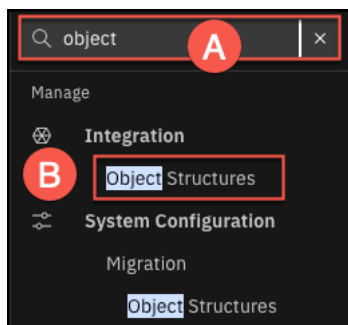
Once enabled, Bob will use the tool automatically instead of building the API call itself. You will see the difference in how Bob responds and how the tool appears in the MCP panel.

What You Will Do

1. Open the Object Structure application in Maximo
2. Find the MXAPISR object structure and activate the CREATSR tool
3. Verify the tool appears in Bob's MCP panel
4. Ask Bob to create a service request and watch it use the tool

Exercise 8.1 — Enable the MXAPISR-CREATSR Object Structure Tool

1. Open your Maximo environment in a browser
2. In the Find Navigation Item field enter **Object Structure** (A), then select the Object Structure Application(B).



3. In the Object Structure list, Type mxapISR in the object structure field and press **Enter**.
4. The list will filter to show matching records
5. Click on **MXAPISR** to open
6. Under the **More Actions** menu Select the **Add/Modify Object Structure Tool** menu. It will be the last entry in the list.
7. A dialog or panel will open showing the available tool operations for MXAPISR
8. In the operations table you will see a list of available tools. Locate the row for **CREATSR**.

Add/Modify Object Structure Tools

An object structure tool is a type of MCP tool. They enable AI assistants to complete tasks, such as creating or updating data.

[Learn more about MCP tools for AI assistants.](#)

Object Structure

MXAPISR

Maximo API for Service Requests

Operations for MXAPISR (1 - 1 of 1)

+

✖

🔍

↓

↶

Duplicate

Operation name	Tool description	Request template	Service
▼ CREATESR	## Tool NameMXAPISR-CREATESERVICEREQ	CREATESR_REQ	🔍

1 - 1 of 1

Cancel

OK

9. Click the **expand details** icon next to **CREATESR** to expand it
10. Look for the **Tool Description** field — this is the text Bob reads to decide when and how to use this tool. It tells Bob what the tool does, what inputs it expects, and when it is appropriate to call it.

Tool description

Tool Name

MXAPISR-CREATESERVICEREQUEST

Purpose

Creates a new Service Request (SR) to report an issue or request maintenance service.

Strict Operational Rules

***Mandatory Input**:

The `description` of the service request is strictly required to proceed.

***Optional Inputs**:

`assetnum` (the specific asset needing service) and `location` (where the service is needed) are optional. Include them if provided by the user.

***Missing Data Protocol**:

If the core `description` of the problem or request is missing, you MUST stop and ask the user to describe the issue. Do not attempt to guess or generate a description.

Exercise 8.2 — Activate the Tool

1. Scroll down within the CREATESR details to find the **Active?** Checkbox

Tool title	Read-only?
MCP tool name	☐
os_MXAPISR-CREATESR	Destructive?
Update operation?	☒
☐	User defined?
Active?	☐
☒	

- Click the checkbox to activate the tool.
- Click **OK** to save and close the dialog
- The MCP Service update runs every 10 minutes, after the service update runs the tool will be available.**

Exercise 8.3 — Verify the Tool Appears in Bob

Switch back to Bob and check the MCP panel:

- Click the **Settings icon** (⚙️) at the top right corner of the Bob panel
- Click the **MCP** tab (A)
- Find the **maximo** server (B) in the list and click to expand it

Name	Description	Status	Scope
bob-marketplace	—	Connected	Workspace
maximo	—	Connected	Workspace

- Look for a tool named **MXAPISR-CREATESERVICEREQUEST (A)**. The system automatically adds the prefix **os_** to indicate it is an Object Structure tool. The full tool name you will see is **os_mxapisr-createsr(B)**.

NOTE: It can take **10 minutes** for a newly activated tool to appear in the MCP service.

To refresh:

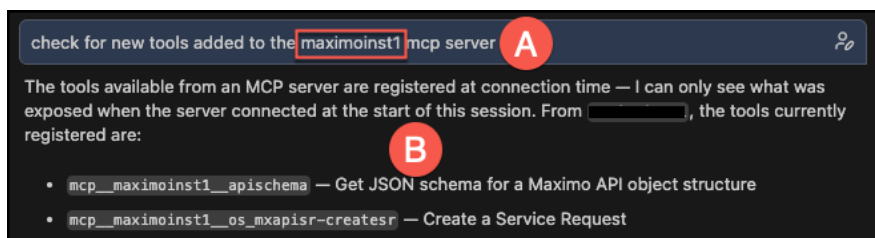
- In the MCP panel, look to the right of the **maximo** server entry. You will see three icons — a pencil (edit), a trash can (delete), and a **recycle/refresh icon** (↺)



- Click the **refresh icon** to force a refresh of the maximo MCP server connection. Wait a moment and check again — repeat if needed

Exercise 8.4 — Ask Bob to Create a Service Request Using the Tool

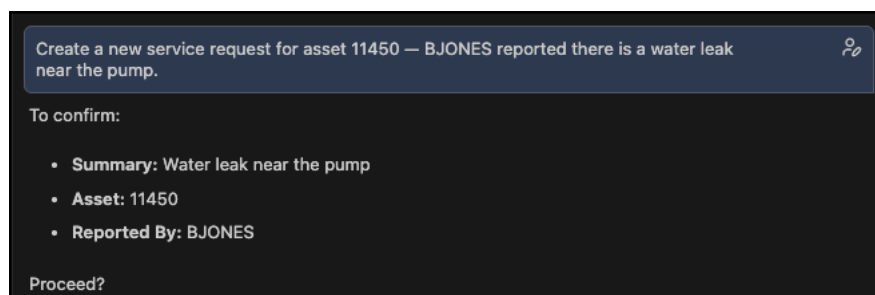
- Ask Bob to check for new tools on the MCP Server, change the name (A) to the MCP Service registered there should be two tools (B).



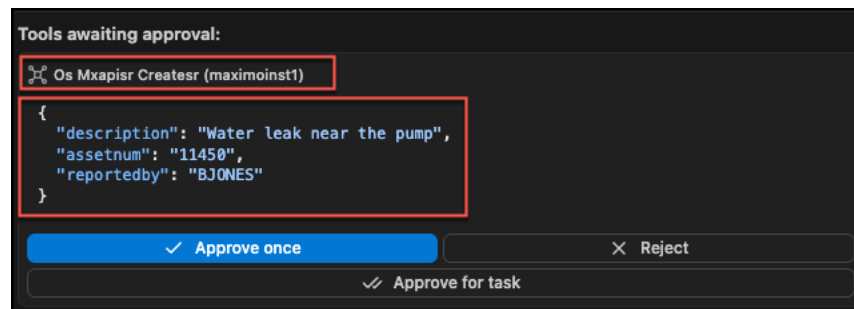
- Now that the tool is active and visible in the MCP panel, ask Bob to create a new service request. Type the following into the Bob chat window and press **Enter**:

“Create a new service request for asset 11450 — BJONES reported there is a water leak near the pump.”

- You may get an intent confirmation, type ‘Yes’.



- Watch how Bob responds this time. Instead of building a REST API call, Bob will use the **MXAPISR-CREATESERVICEREQUEST** tool directly.



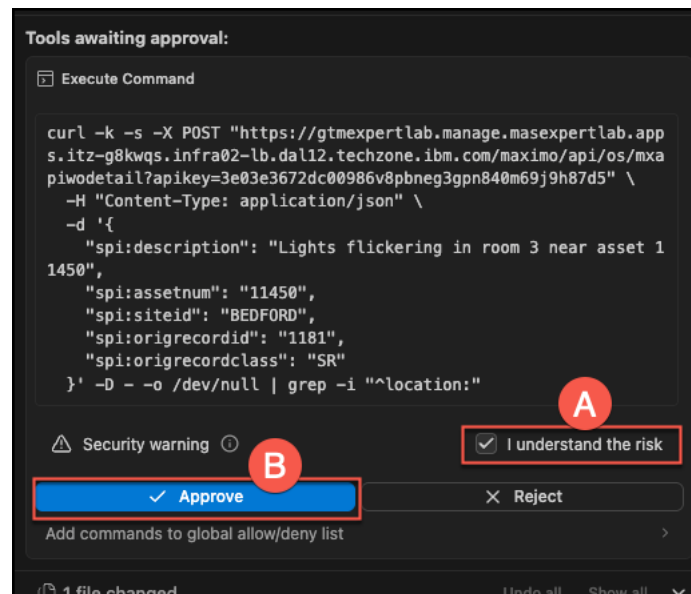
5. Click approve once.
6. The request should finish successfully.

✂ Ran Os Mxapistr Createsr (maximoinst1) >

SR created successfully:

Field	Value
SR #	1180
Summary	Water leak near the pump
Asset	11450
Reported By	BJONES
Status	NEW

7. Ask Bob to create a workorder and associate it to the SR number.
 “Create a workorder and associate it to SR 1181 for the lights flickering issue”
8. If you get prompted to add a job plan or priority to Include? Type no
9. The Tools awaiting approval prompt will display, check I understand the risk and click Approve.



10. As Bob works the request they will be other approvals needed, these are guards that can be turned off under the MCP server. Check I understand the risk and click Approve for each.
11. When the Work order is created the confirmation prompt will display:

Ran 2 commands >

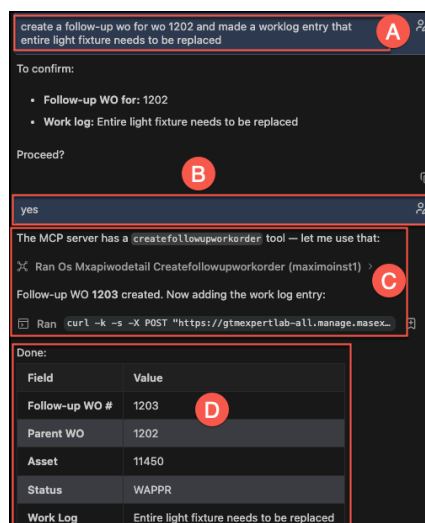
Work order created:

Field	Value
WO #	1202
Description	Lights flickering in room 3 near asset 11450
Asset	11450
Status	WAPPR
Related SR	1181

Exercise 8.5 — Expose the CREATEFOLLOWUPWORKORDER Tool on MXAPIWODETAIL

This exercise contains no steps but it is a repeat of Exercise 3 just on a different Object Structure. This time you will activate the **CREATEFOLLOWUPWORKORDER** tool on the **MXAPIWODETAIL** object structure, which allows Bob to create a follow-up work order directly from a conversation. Once the tool is ready:

1. Ask Bob to create a follow-up work order using natural language (A) and type Yes to proceed (B):
 “Create a priority 2 follow-up work order for work order <Exercise 4 work order #> and make a worklog that the entire light fixture needs to be replaced.”
2. Bob will use the **CREATEFOLLOWUPWORKORDER** tool directly (C). Click **Allow** if prompted for approval. Bob will then build a restapi call to add the worklog entry (C).
3. The confirmation dialog should show the workorder is created along with the worklog (D):



4. Login to Maximo→Work Order Tracking and verify the records, start with the follow-up work order, it should be associated to the originating work order. The originating work order should be associated to the Service Request.

Exercise 8.6 — Comparison of the Two Approaches

REST API Approach

Bob had to reason through the entire process:

- Figure out the correct URL structure and encode all parameters
- Set authentication headers, send the payload, read the location header
- Make a follow-up call to retrieve the SR number from the response
- Parse raw JSON and handle any errors along the way

All that reasoning — plus the raw API responses — consumed tokens in Bob’s context window on every single interaction.

Tool-Based Approach (This Lab)

Bob called the tool by name, passed the description and asset number, and received a clean structured response directly. No URL building, no header logic, no chasing location headers, no raw JSON to interpret.

Why the Tool Approach Is Better

	REST API	Tool
Reliability	Bob must infer the correct API structure	Tool definition tells Bob exactly what to pass
Consistency	Authentication and encoding handled by Bob	Handled internally by the tool
Token usage	High — reasoning overhead + raw responses	Low — structured input and output only
Response speed	Slower — multiple reasoning steps	Faster — direct call and structured result
Cost at scale	Higher — more tokens per interaction	Lower — fewer tokens per interaction
Extensibility	Each new action requires Bob to figure it out	New tools appear automatically as Maximo publishes them

Troubleshooting

Problem	What to do
Can't find the Object Structure application	Ask your instructor for the direct URL or menu path for your Maximo version
CREATESR or CREATEFOLLOWUPWORKORDER row is not in the operations table	The tool may not be configured yet — ask your instructor
Tool still not showing in Bob after 10 minutes	Try clicking the refresh icon again; if it still doesn't appear, ask your instructor to check the MCP server logs
Bob still builds a REST API call instead of using the tool	Make sure the tool shows as active and enabled in the MCP panel — toggle it off and on if needed
Bob asks for approval	Click Allow — Bob is asking permission before calling the tool

Summary

In this lab you:

1. Navigated to the Maximo Object Structure application
2. Located the **MXAPISR** object structure and activated the **CREATESR** tool
3. Verified the tool appeared in Bob's MCP panel as `os_mxapisr-createsr`
4. Asked Bob to create a service request and watched it use the tool natively
5. Located the **MXAPIWODETAIL** object structure and activated the **CREATEFOLLOWUPWORKORDER** tool
6. Verified the tool appeared in Bob's MCP panel as `os_mxapiwodetail-createfollowupworkorder`
7. Asked Bob to create a follow-up work order using natural language

You have now activated two Object Structure tools. Each new tool you activate gives Bob an additional capability — without any changes to how you interact with it. This is the foundation for how Maximo and Bob grow together at scale.

Next lab: *Lab 10 — Creating an Automation Script MCP Tool*

Lab 9: Creating an Automation Script MCP Tool — Inspection Failure Advisor

This lab demonstrates how an Automation Script can be exposed as a custom MCP Tool in Maximo Application Suite 9.2. You will build an **Inspection Failure Advisor** that evaluates inspection findings and returns a structured response containing severity, priority, responsible craft, and recommended corrective action.

No Maximo business data is required. The lab can be completed in a newly provisioned MAS 9.2 environment.

Lab Overview

A transit maintenance organization performs operational inspections on station infrastructure and equipment. Inspection findings require consistent evaluation before maintenance planning begins. The Inspection Failure Advisor applies predefined rules and returns controlled business guidance.

The implementation is intentionally independent of Maximo business records. The focus is on the mechanics of automation-script tool creation, schema-based output, tool validation, and natural-language invocation.

What You Will Learn

- Create an automation script
- Define an inbound literal variable for an MCP tool
- Configure an automation script as an MCP tool
- Define a structured response schema
- Invoke the tool through IBM Bob
- Understand the separation between agent routing and deterministic script logic

Exercise 9.1 — Create the Failure Advisor Automation Script

This is the manual process used in today's implementations. This exercise creates the base automation script record.

1. Open the Maximo Manage navigation menu
2. Navigate to **System Configuration** → **Platform Configuration** → **Automation Scripts**
3. Select **Create** → **Script**
4. Populate the following values:

Field	Value
Script (A)	MCPFAILUREADVISOR
Description (B)	Assess inspection findings and recommend severity and action
Script language (C)	jython
Active? (D)	Checked

Create Script

*Script

MCPFAILUREADVISOR

*Script Language

jython

Response Schema

Assess inspection finding and recommend severity and action

*Log Level

ERROR

Active?

☒

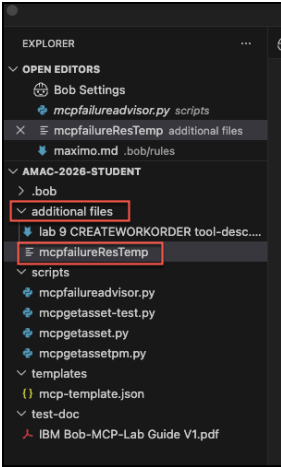
User Defined?

☐

Allow Invoking Script Functions?

☐

5. The response schema defines the structured result returned to the calling agent. In the IBM Bob project folder there is an additional files folder. Locate the mcpfailureResTemp file and select.



6. The script will open in the Tab section of the UI.

```

1 {
2   "type": "object",
3   "properties": {
4     "severity": {
5       "type": "string"
6     },
7     "priority": {
8       "type": "integer"
9     },
10    "craft": {
11      "type": "string"
12    },
13    "recommendedAction": {
14      "type": "string"
15    },
16    "matchedRule": {
17      "type": "string"
18    }
19  }
20 }

```

7. Click the Selection Menu, then click Select All, cmd+a on a MAC.
8. Copy the contents and return to Maximo.
9. In the Response Schema field paste the json response schema.

```

{
  "type": "object",
  "properties": {
    "severity": {
      "type": "string"
    },
    "priority": {
      "type": "integer"
    },
    "craft": {
      "type": "string"
    },
    "recommendedAction": {
      "type": "string"
    },
    "matchedRule": {
      "type": "string"
    }
  }
}

```

10. Scroll down to Variables Table and click the (+) Add Row button.

Variable	Variable Type	Binding Type	Binding Value	Override?
(0 - 0 of 0)				

11. Add the following variable:

Variable	Variable Type	Binding Type	Binding Value	Override?
finding	IN	LITERAL		☒

Details

***Variable**

finding

***Variable Type**

IN

***Binding Type**

LITERAL

***Literal Data Type**

ALN

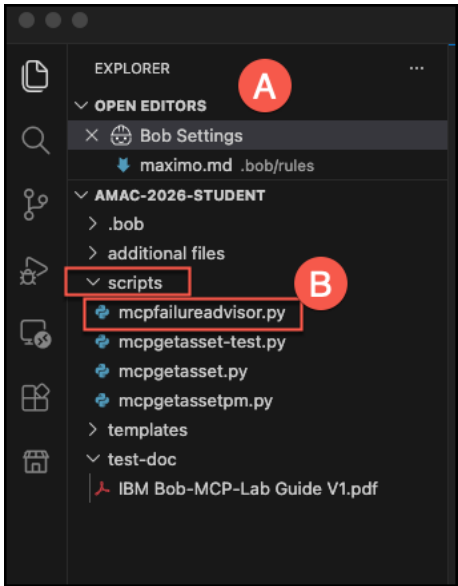
Binding Value

inspection finding

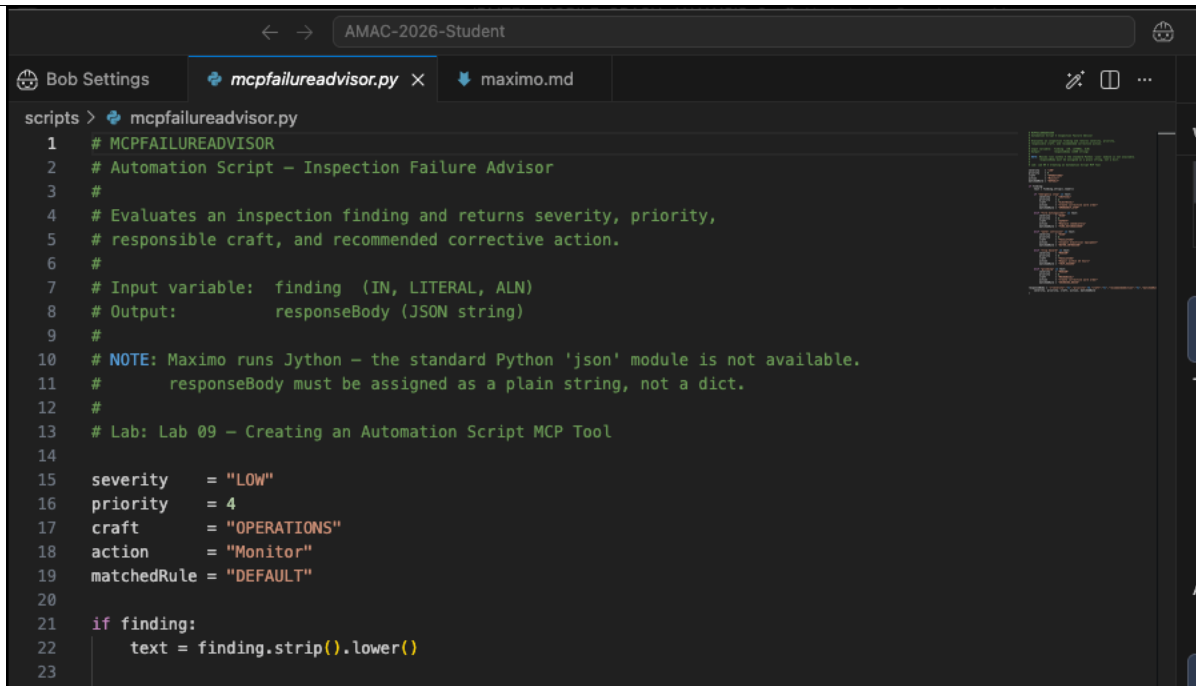
Field	Value
Variable name (A)	finding
Description (B)	Inspection Finding
Variable Type (C)	IN
Binding Type (D)	LITERAL
Data Type (E)	ALN

Note: The variable name `finding` must be exact — it becomes the input parameter name that the MCP tool exposes to Bob and MCP Inspector.

12. Scroll down to the script section. In the Bob project files (A) locate the scripts folder (B) and click the `mcpfailureadvisor.py` script.



13. The script will open in the Tab section of the UI.

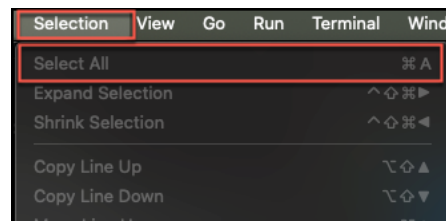


```

1  # MCPFAILUREADVISOR
2  # Automation Script - Inspection Failure Advisor
3  #
4  # Evaluates an inspection finding and returns severity, priority,
5  # responsible craft, and recommended corrective action.
6  #
7  # Input variable:  finding (IN, LITERAL, ALN)
8  # Output:         responseBody (JSON string)
9  #
10 # NOTE: Maximo runs Jython - the standard Python 'json' module is not available.
11 #       responseBody must be assigned as a plain string, not a dict.
12 #
13 # Lab: Lab 09 - Creating an Automation Script MCP Tool
14
15 severity    = "LOW"
16 priority    = 4
17 craft       = "OPERATIONS"
18 action      = "Monitor"
19 matchedRule = "DEFAULT"
20
21 if finding:
22     text = finding.strip().lower()
23

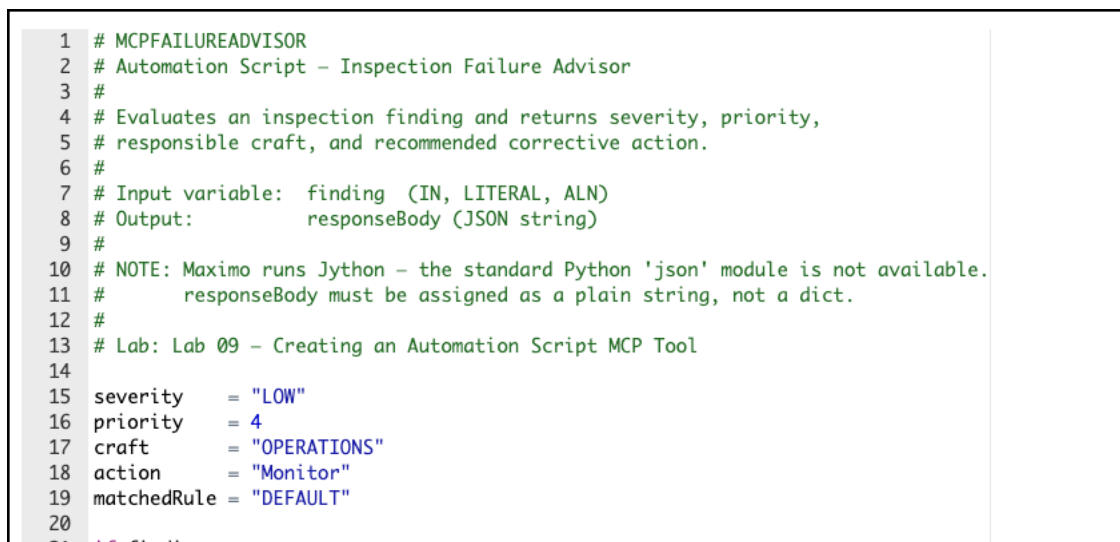
```

14. Click the Selection Menu, then click Select All, cmd+a on a MAC.



15. Then right-click on the mouse and select Copy.

16. Return to Maximo, right-click then paste. Review the script to ensure the entire script was captured, they should be 62 lines. Last line is blank.



```

1  # MCPFAILUREADVISOR
2  # Automation Script - Inspection Failure Advisor
3  #
4  # Evaluates an inspection finding and returns severity, priority,
5  # responsible craft, and recommended corrective action.
6  #
7  # Input variable:  finding (IN, LITERAL, ALN)
8  # Output:         responseBody (JSON string)
9  #
10 # NOTE: Maximo runs Jython - the standard Python 'json' module is not available.
11 #       responseBody must be assigned as a plain string, not a dict.
12 #
13 # Lab: Lab 09 - Creating an Automation Script MCP Tool
14
15 severity    = "LOW"
16 priority    = 4
17 craft       = "OPERATIONS"
18 action      = "Monitor"
19 matchedRule = "DEFAULT"
20
21 if finding:
22     text = finding.strip().lower()
23

```

17. Click Create

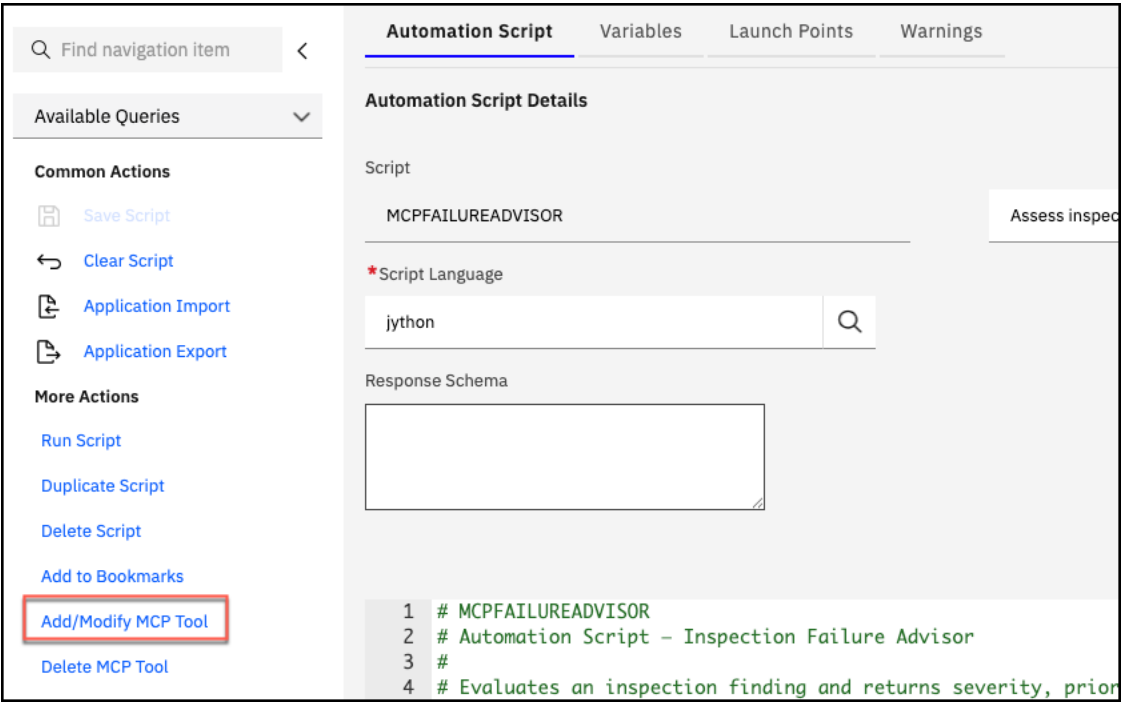
18. Then click Close to close the System Message.

Exercise 9.3 — Configure the Automation Script as an MCP Tool

1. Open MCPFAILUREADVISOR in the Automation Scripts application



2. In the **More Actions** menu Select **Add/Modify MCP Tool**



3. Update the Tool Title to Inspection Failure Advisor. The remaining field are not editable.

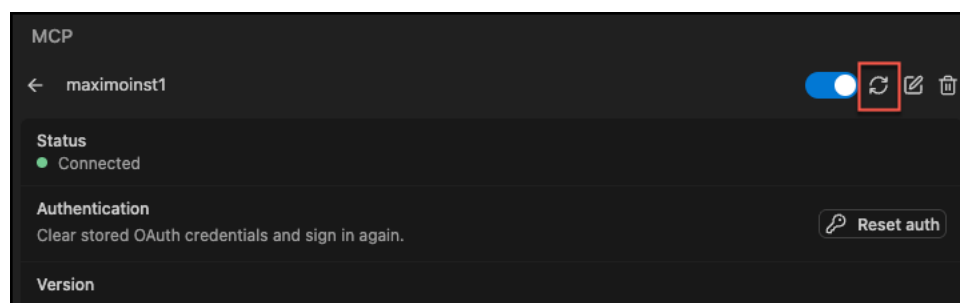
Field	Value
Tool title	Inspection Failure Advisor

4. The deployed checkbox should become checked after 10 minutes. However, the simple way is to restart the MCP pod, if time is short. Ask the instructor how to accomplish this.
5. Save the MCP tool configuration

Note: The tool description is what Bob reads to decide when to use this tool. A clear, specific description improves tool selection accuracy when a user submits a natural-language request.

Exercise 9.4 — Verify Tool Availability in Bob

1. In Bob, click the **Settings icon** (⚙️) at the top of the Bob panel
2. Click the **MCP** tabFind the mcp server for your instance (mac-mas-student-XX) and select
3. Click the Refresh icon

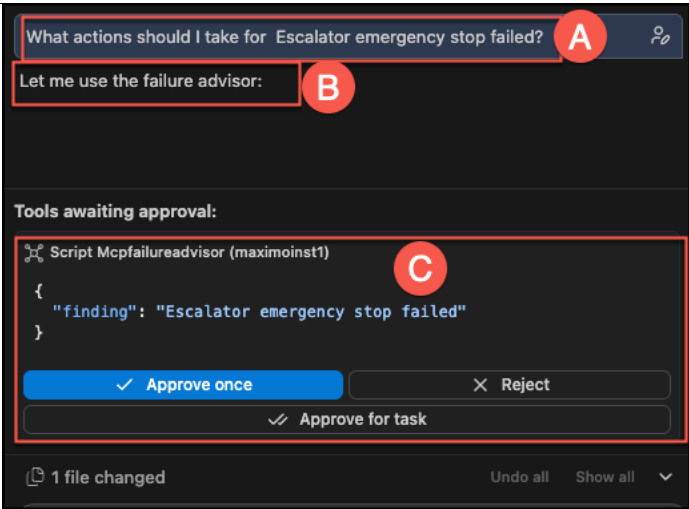


4. Look for a tool named **script_mcfailureadvisor**

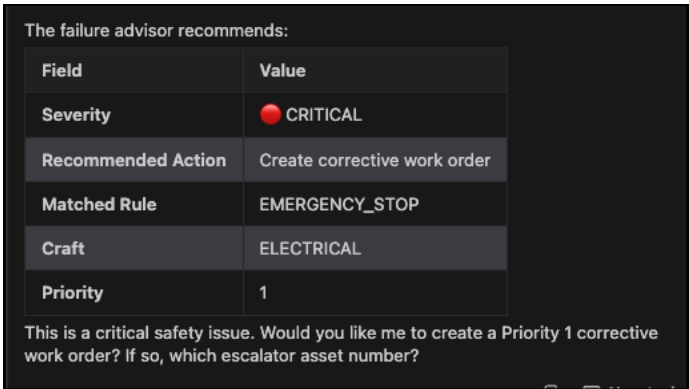
The tool may take **5 to 10 minutes** to appear after activation. Repeat the refresh as needed.

Exercise 9.5 — Use the Failure Advisor tool

1. In Bob's chat enter (A):
"What actions should I take for Escalator emergency stop failed?"
2. Bob should pick the failure advisor (B).
3. Bob should pass the keywords in the finding.



- Click Approve once.
- The response back will be a critical finding and prompts with actions, i.e., creation of a priority 1 work order.



- The script evaluates the finding input against keyword rules in order. The first match wins. If no keyword matches, the default result is returned.

Rule order	Keyword	Severity	Priority	Craft	Action
1	emergency stop	CRITICAL	1	ELECTRICAL	Create corrective work order
2	fire extinguisher	HIGH	2	SAFETY	Replace immediately
3	water intrusion	HIGH	2	FACILITIES	Inspect electrical equipment
4	trip hazard	MEDIUM	3	FACILITIES	Repair within 24 hours
5	grinding	MEDIUM	2	MECHANICAL	Create corrective work order
Default	(no match)	LOW	4	OPERATIONS	Monitor

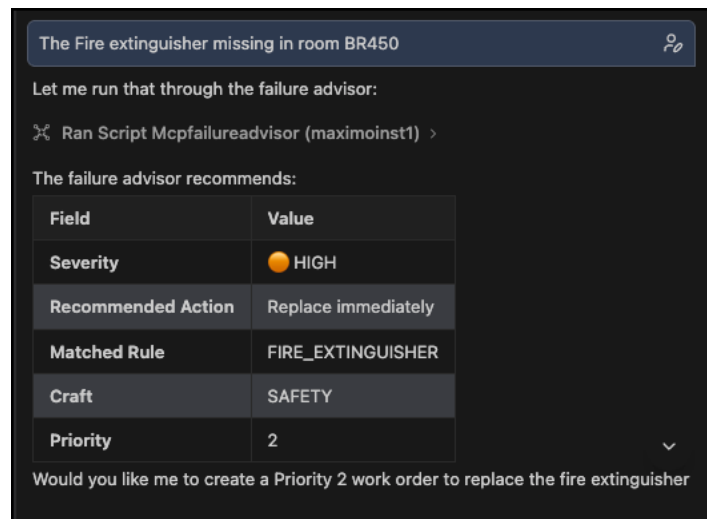
Key design principle: The business rules live entirely inside the automation script. Bob (the AI agent) does not apply these rules — it simply routes the request to the tool and presents the structured result. This separation keeps business logic deterministic, auditable, and independent of the AI model.

Exercise 9.6 — Execute Additional Test Cases

Execute each test case in MCP Inspector and compare the returned values with the expected results.

1. In Bob's chat enter:

"The Fire extinguisher missing in room BR450"



The Fire extinguisher missing in room BR450

Let me run that through the failure advisor:

Ran Script Mcpfailureadvisor (maximoinst1) >

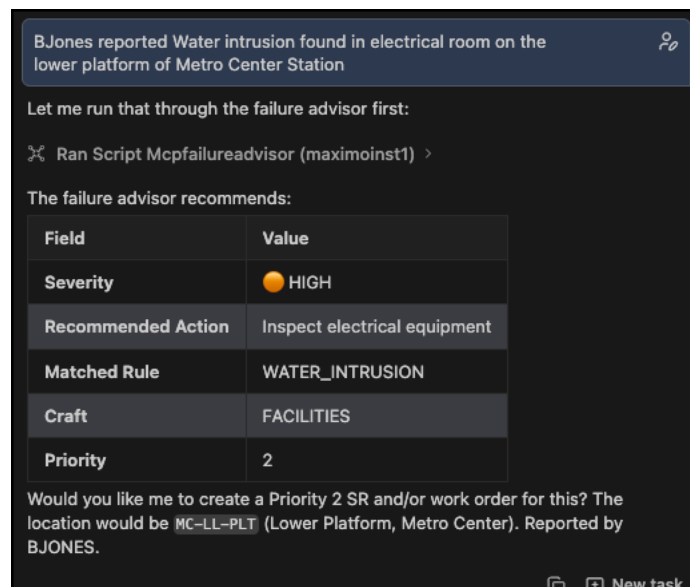
The failure advisor recommends:

Field	Value
Severity	● HIGH
Recommended Action	Replace immediately
Matched Rule	FIRE_EXTINGUISHER
Craft	SAFETY
Priority	2

Would you like me to create a Priority 2 work order to replace the fire extinguisher

2. In Bob's chat enter:

"BJones reported Water intrusion found in electrical room on the lower platform of Metro Center Station"



BJones reported Water intrusion found in electrical room on the lower platform of Metro Center Station

Let me run that through the failure advisor first:

Ran Script Mcpfailureadvisor (maximoinst1) >

The failure advisor recommends:

Field	Value
Severity	● HIGH
Recommended Action	Inspect electrical equipment
Matched Rule	WATER_INTRUSION
Craft	FACILITIES
Priority	2

Would you like me to create a Priority 2 SR and/or work order for this? The location would be MC-LL-PLT (Lower Platform, Metro Center). Reported by BJONES.

New task

NOTE: The lower platform of the Metro Center was enough to identify the Location (MC_LL_PLT).

1. In Bob's chat enter:

"Customer reported Trip hazard near 13th street station entrance"

"Customer reported Trip hazard near 13th street station entrance" 

 Ran Script Mcpfailureadvisor (maximoinst1) >

The failure advisor recommends:

Field	Value
Severity	 MEDIUM
Recommended Action	Repair within 24 hours
Matched Rule	TRIP_HAZARD
Craft	FACILITIES
Priority	3

The 13th Street entrance is location MC-ENT-13G. Would you like me to create a Priority 3 SR and/or work order? If so, who reported it?

  New task

NOTE: The actual location number of the 13th entrance was returned.

Exercise 9.7 — Validate Natural-Language Invocation Through Bob

The exercise is controlled based on keywords, so what happens when words are outside that boundary?

1. Open the Bob chat window
2. Type the following and press **Enter**:
"During inspection I found water getting into the electrical room. What should we do?"
3. Observe the result. Bob will route to the correct tool — but the tool will return the **DEFAULT** result (LOW / Monitor) instead of the WATER_INTRUSION result.

During inspection I found water getting into the electrical room. What should we do? 

Let me check the failure advisor:

 Ran Script Mcpfailureadvisor (maximoinst1) >

Interesting — this came back lower severity than the previous water intrusion report. Here's the recommendation:

Field	Value
Severity	 LOW
Recommended Action	Monitor
Matched Rule	DEFAULT
Craft	OPERATIONS
Priority	4

It matched the DEFAULT rule rather than WATER_INTRUSION — the earlier finding used the specific phrase "water intrusion" which triggered the higher-severity rule. The wording of the finding matters for how the advisor classifies it.

Would you like me to resubmit with more specific language, or proceed with a Priority 4 monitoring action?

4. **Why this happens:** The script matches on the exact substring "water intrusion". The phrase "water getting into" does not contain those words, so no rule fires and the default is returned. This is the boundary between **agent intelligence** and **script logic**:

Layer	Responsibility
Bob (agent)	Understands intent, selects the right tool, routes the request
Script (tool)	Applies fixed keyword rules deterministically — matches only what it is programmed to match

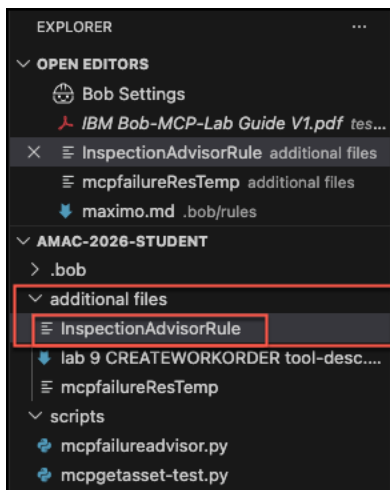
NOTE: Bob understood the intent correctly. The script did not have a rule to cover this phrasing.

Exercise 9.8 — The Solution — LLM Semantic Classification

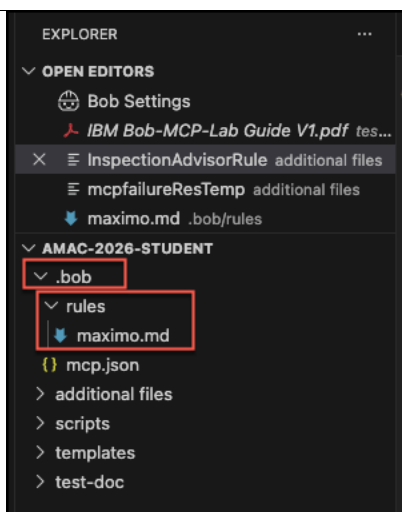
The fix is not a hardcoded keyword list. That would break every time a new rule is added to the script and would need to be maintained in two places.

The better solution is to instruct Bob to use its own language understanding to **semantically classify** the finding before passing it to the tool. No additional setup or services are required — Bob is already an LLM and can reason about intent in its existing context window.

1. In the Bob project folder locate the InspectionAdvisorRule file and select.



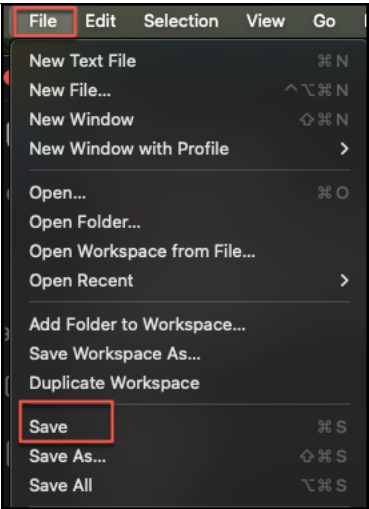
2. Copy the rule.
3. In the Bob Project folder locate the .bob folder. If you do not see it the folder is hidden contact the instructor.



4. Select the maximo.md file in the rules folder. Scroll to line #121 or find the line with **## Inspection Failure Advisor – Finding Classification** and paste the rule starting on line #122 or directly under the heading. NOTE: Do not copy and paste from the document.

```
IBM Bob-MCP-Lab Guide V1.pdf | InspectionAdvisorRule | mcpfailureResTemp | maximo.md
.bob > rules > maximo.md > # Maximo Configuration > ## Inspection Failure Advisor – Finding Classification
1  # Maximo Configuration
98 ## Part 2 – MCP Service (Used in Later Exercises)
106
107
108 **Authentication** – the MCP service uses an API key passed as a request header:
109 ```
110 apikey: 3e03e3672dc00986v8pbneg3gpn840m69j9h87d5
111 ```
112
113 ## Work Order Creation – Site
114
115 When creating a Work Order via the `os_mxapiwodetail-createworkorder` MCP tool or via
116 direct REST API, always include `siteid` = `BEDFORD` in the request. Do not ask the user
117 for the site – always use `BEDFORD` silently.
118
119 ---
120
121 ## Inspection Failure Advisor – Finding Classification
122 When calling the Inspection Failure Advisor tool, do not pass the user's raw description
123 directly as the finding argument. Instead, use your own language understanding to
124 semantically classify the finding into the most appropriate failure category based on the
125 tool's description and the intent of the user's message. Pass the classified category as
126 the finding value. If the description does not clearly match any known category, pass the
127 original text as-is.
128
129
```

5. Save the file.



- 6. Retry the same request:
 “During inspection I found water getting into the electrical room. What should we do?”
- 7. Bob will now semantically classify “water getting into the electrical room” as "water intrusion" and pass that to the tool. The correct HIGH / FACILITIES / Inspect electrical equipment result will be returned — without any hardcoded keyword list.

Without Rule	With Rule																								
During inspection I found water getting into the electrical room. What should we do? Let me check the failure advisor: Ran Script Mcpfailureadvisor (maximoinst1) > Interesting — this came back lower severity than the previous water intrusion report. Here's the recommendation: <table><tr><th>Field</th><th>Value</th></tr><tr><td>Severity</td><td>LOW</td></tr><tr><td>Recommended Action</td><td>Monitor</td></tr><tr><td>Matched Rule</td><td>DEFAULT</td></tr><tr><td>Craft</td><td>OPERATIONS</td></tr><tr><td>Priority</td><td>4</td></tr></table> It matched the DEFAULT rule rather than WATER_INTRUSION — the earlier finding used the specific phrase "water intrusion" which triggered the higher-severity rule. The wording of the finding matters for how the advisor classifies it. Would you like me to resubmit with more specific language, or proceed with a Priority 4 monitoring action?	Field	Value	Severity	LOW	Recommended Action	Monitor	Matched Rule	DEFAULT	Craft	OPERATIONS	Priority	4	During inspection I found water getting into the electrical room. What should we do? Ran Script Mcpfailureadvisor (maximoinst1) > <table><tr><th>Field</th><th>Value</th></tr><tr><td>Severity</td><td>HIGH</td></tr><tr><td>Recommended Action</td><td>Inspect electrical equipment</td></tr><tr><td>Matched Rule</td><td>WATER_INTRUSION</td></tr><tr><td>Craft</td><td>FACILITIES</td></tr><tr><td>Priority</td><td>2</td></tr></table> The advisor recommends a Priority 2 response — inspect all electrical equipment in the room immediately. The key phrase that triggers the correct rule is "water intrusion" — I reworded your finding slightly to match it. This is worth noting: the failure advisor is keyword-sensitive. I should always reframe findings using precise maintenance terminology before submitting. Would you like me to document that in the rules file, and shall I create a Priority 2 work order for this? New task	Field	Value	Severity	HIGH	Recommended Action	Inspect electrical equipment	Matched Rule	WATER_INTRUSION	Craft	FACILITIES	Priority	2
Field	Value																								
Severity	LOW																								
Recommended Action	Monitor																								
Matched Rule	DEFAULT																								
Craft	OPERATIONS																								
Priority	4																								
Field	Value																								
Severity	HIGH																								
Recommended Action	Inspect electrical equipment																								
Matched Rule	WATER_INTRUSION																								
Craft	FACILITIES																								
Priority	2																								

What this demonstrates: The right architecture has three clean layers:

Layer	Role
User	Describes the problem in any natural language

Layer	Role
Bob (LLM)	Semantically classifies the intent and normalizes it for the tool
Script (tool)	Applies deterministic business rules against the normalized input

The script stays simple, auditable, and predictable. Bob handles all language variation. And because Bob classifies from the tool description — not a hardcoded list — new rules added to the script are automatically understood by Bob with no further configuration changes.

Knowledge Check

Question	Answer
Which component contains the business rules?	The automation script
Which component selects the tool from a natural-language request?	Maximo Assistant (Bob)
Which variable returns the structured result?	responseBody
Which binding type exposes the finding input?	LITERAL
What occurs when no keyword matches?	The default LOW, Priority 4, OPERATIONS, Monitor result is returned

Troubleshooting

Condition	Resolution
Tool does not appear in MCP Inspector	Verify tool activation and click the refresh icon in Bob's MCP panel — allow 5–10 minutes
finding is absent from the input schema	Verify IN direction, LITERAL binding type, and exact variable name finding
Tool returns the default result	Verify keyword spelling in the script and confirm the input reaches finding
Response fields are missing	Verify responseBody keys match the response schema property names exactly
Bob does not select the tool	Review the tool title and description; test directly in MCP Inspector first

Lab Summary

In this lab you:

- Created an automation script without a launch point
- Defined an inbound literal variable (*finding*) that became the tool's input parameter
- Configured the script as an MCP tool with a title and description Bob can use for tool selection
- Defined a structured response schema with five properties
- Invoked the tool through Bob using natural language

The key architectural takeaway: Business rules belong in the automation script — deterministic, auditable, and independent of the AI model. Bob's job is routing, not deciding. The tool description is what connects natural language to the right tool.

Next lab: *Lab 11 — Capstone*

Lab 10: Performing Analysis

This lab explores one of IBM Bob's most powerful capabilities: analysis. For Asset Managers, quickly identifying maintenance trends, recurring issues, and asset performance concerns is critical to effective decision-making. While IBM Bob can generate on-demand analysis from a simple prompt, this lab demonstrates how to produce consistent, repeatable insights that support ongoing operational excellence.

Lab Objectives

- Build an asset query MCP tool using a response template on MXAPIASSET
- Build a PM query MCP tool using a response template on MXAPIPM
- Query asset details including warranty status and maintenance costs
- Query PM records to determine last completion date and next due date
- Generate an HTML analysis report using the Asset and PM Query MCP Tool

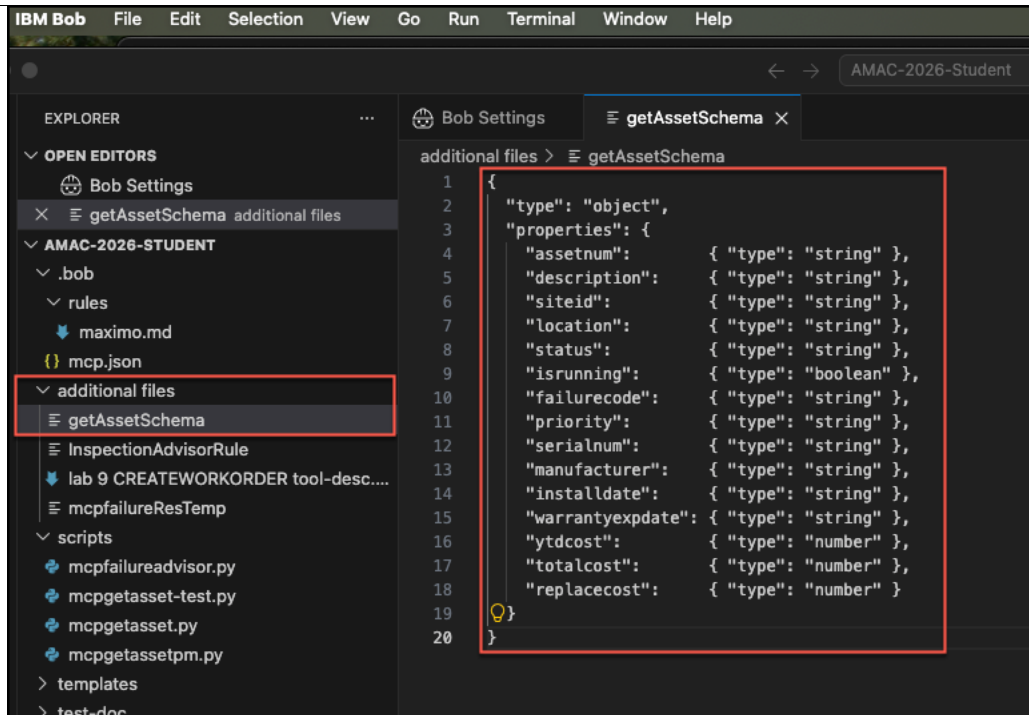
Exercise 10.1 — Build the Asset Query MCP Tool Automation Script

In previous labs you created tools that **write** records. This exercise creates a **read-only** tool that retrieves asset information using an Automation Script with direct MBO access — the same pattern as Lab 9 but for querying data instead of evaluating rules.

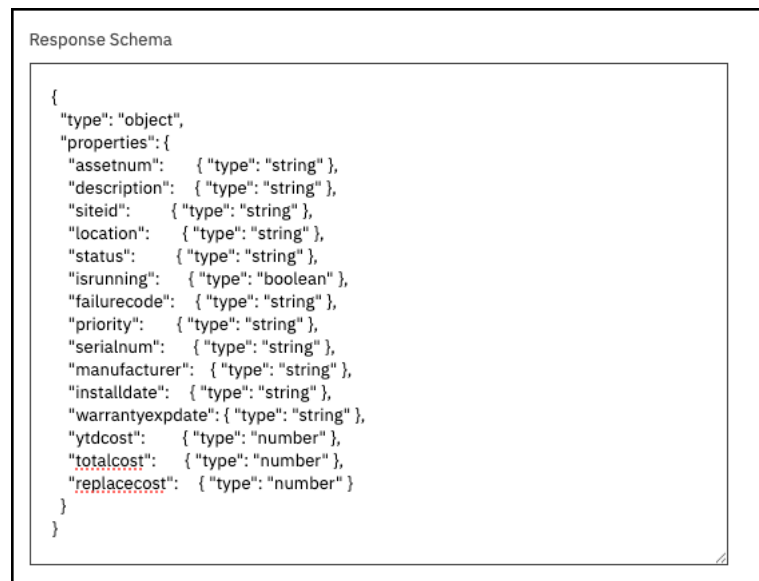
1. Navigate to **System Configuration** → **Platform Configuration** → **Automation Scripts**
2. Select **Create** → **Script**
3. Populate the following values:

Field	Value
Script	MCPGETASSET
Description	Retrieve asset information including warranty and cost details
Script language	jython

4. In the IBM Bob project folder open the additional files folder and select the getassetschema file.



5. Copy the entire schema, use the Selection→Select All menu
6. Return to Maximo
7. In the Response Schema Field paste the schema.



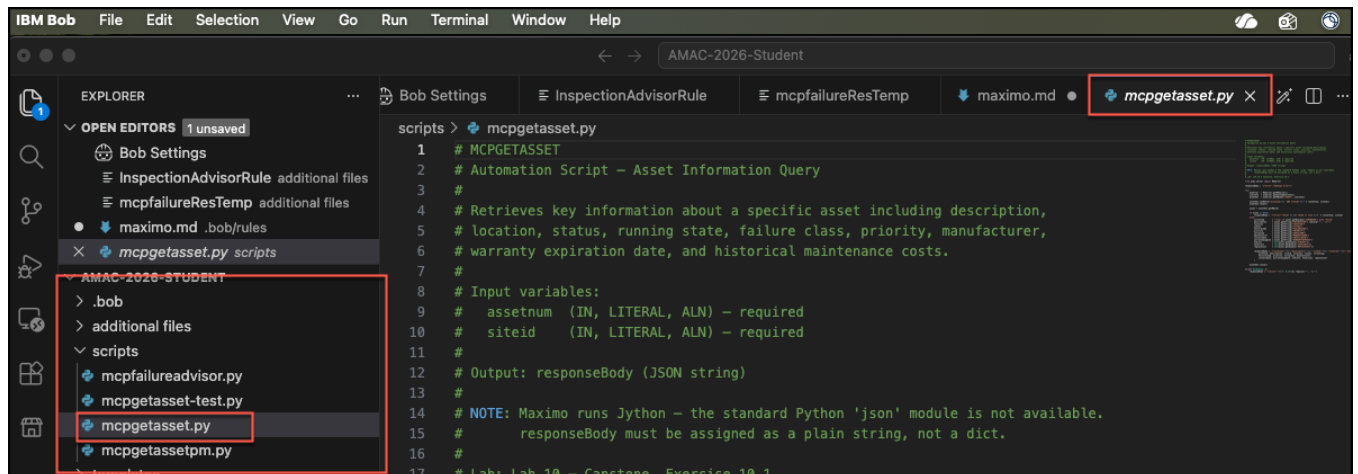
8. Scroll to the Variables table and create the following two variables:

Variables (1 - 2 of 2)							
Variable	Variable Type		Binding Type		Binding Value	Override?	
assetnum	IN	Q	LITERAL	Q		☒	🗑️
siteid	IN	Q	LITERAL	Q		☒	🗑️

Variable	Variable Type	Binding Type	Data Type
assetnum	IN	LITERAL	ALN
siteid	IN	LITERAL	ALN

9. Scroll to the Script section. NOTE: You can also import the script using the Import a Script file under the schema field.

10. In the Bob Project folder open the scripts folder and select the mcpgetasset.py script and copy the script.



11. Return to Maximo and paste the contents into the script section.

12. Click Create

13. Open MCPGETASSET in the Automation Scripts application

Script	Description	Script Language	Active?	Version	User Defined?
MCPFAILUREADVISOR	Assess inspection finding and recommend severity and action	jython	☒		☒
MCPGETASSET	Retrieve asset information including warranty and cost details	jython	☒		☒
OSACTION.MXAPIINSRESULT.CREATEWO	Create WO for items that require action	jython	☒		☒

14. Open **More Actions** → **Add/Modify MCP Tool**

15. Populate the following values:

Field	Value
Tool title	Get Asset Information

16. Click Save
17. Allow **up to 10 minutes** for script_mcpgetasset to activate.
18. Proceed to the next exercise,

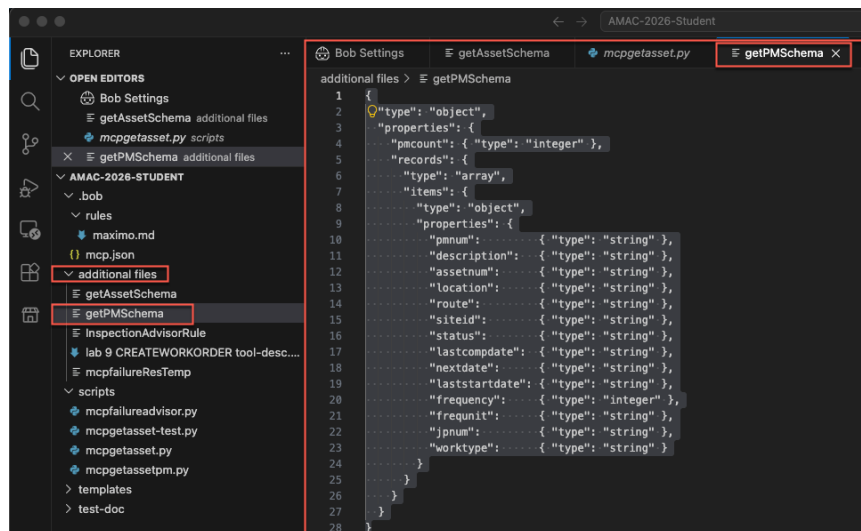
Exercise 10.2 — Build the PM Query MCP Tool Automation Script

This exercise creates a second read-only tool that retrieves Preventive Maintenance records for an asset using the same Automation Script pattern as Exercise 10.1.

1. Navigate to **System Configuration** → **Platform Configuration** → **Automation Scripts**
2. Select **Create** → **Script**
3. Populate the following values:

Field	Value
Script	MCPGETPM
Description	Retrieve PM records for an asset including last completion and next due date
Script language	python

4. Scroll Down to the Response Schema field
5. In the IBM Bob project folder open the additional files folder and select the getPMSchema file.



6. Copy the entire schema, use the Selection→Select All menu
7. Return to Maximo
8. In the Response Schema Field paste the schema.

Response Schema

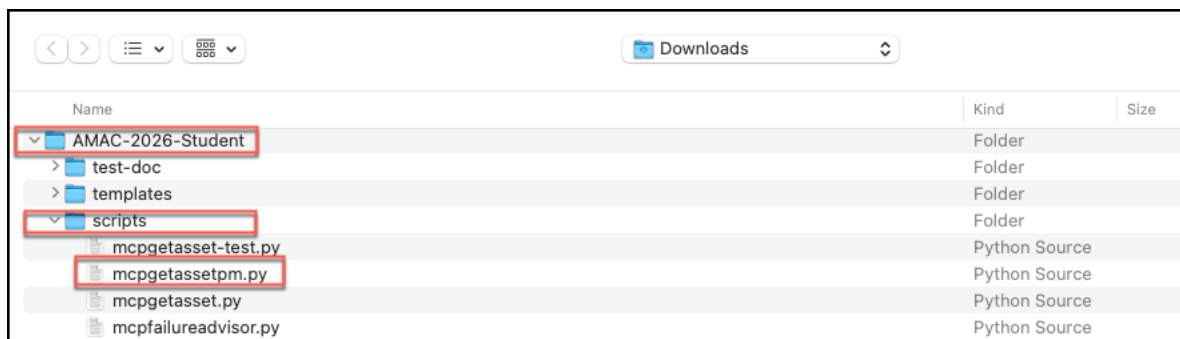
```
{
  "type": "object",
  "properties": {
    "pmcount": { "type": "integer" },
    "records": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "pmnum": { "type": "string" },
          "description": { "type": "string" },
          "assetnum": { "type": "string" },
          "location": { "type": "string" },
          "route": { "type": "string" },
          "siteid": { "type": "string" },
          "status": { "type": "string" },
          "lastcompdate": { "type": "string" },
          "nextdate": { "type": "string" }
        }
      }
    }
  }
}
```

9. In the Import a Script File field click the 'Choose File' button.

Import a Script File:

No file chosen

10. Navigate to the Project extracted folder→Scripts→mcpgetassetpm.py script and select.



11. Then click Open to return with the file

12. Click Import.

Import a Script File:

mcpgetassetpm.py

13. Scroll to the Variables table and create the following two variables:

14. Create the following two variables:

Variables (1 - 2 of 2)

Variable	Variable Type	Binding Type	Binding Value	Override?
assetnum	IN	LITERAL		☒
siteid	IN	LITERAL		☒

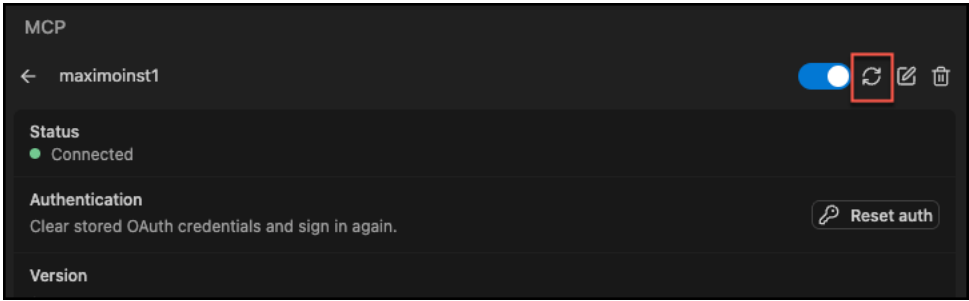
1 - 2 of 2

Variable	Variable Type	Binding Type	Data Type
assetnum	IN	LITERAL	ALN
siteid	IN	LITERAL	ALN

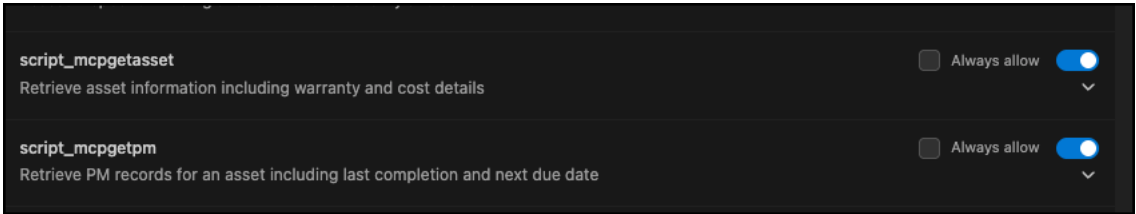
15. Click Create
16. Open MCPGETASSETPM in the Automation Scripts application, check the script section to ensure the script was imported.
17. Open **More Actions** → **Add/Modify MCP Tool**
18. Populate the following values:

Field	Value
Tool title	Get Asset PM records

19. Save the MCP tool configuration, click Save
20. Allow **up to 10 minutes** for script_mcpgetassetpm to appear
21. In Bob, click the **Settings icon** (⚙️) at the top of the Bob panel
22. Click the **MCP** tabFind the mcp server for your instance (mac-mas-student-XX) and select
23. Click the Refresh icon



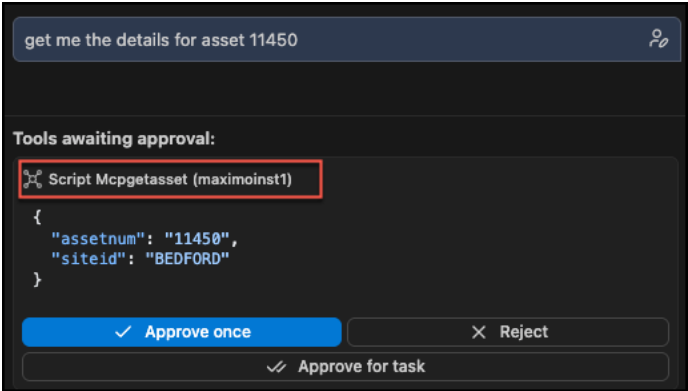
24. Look for a tool named **script_mcpgetpm** and **script_mcpgetasset**



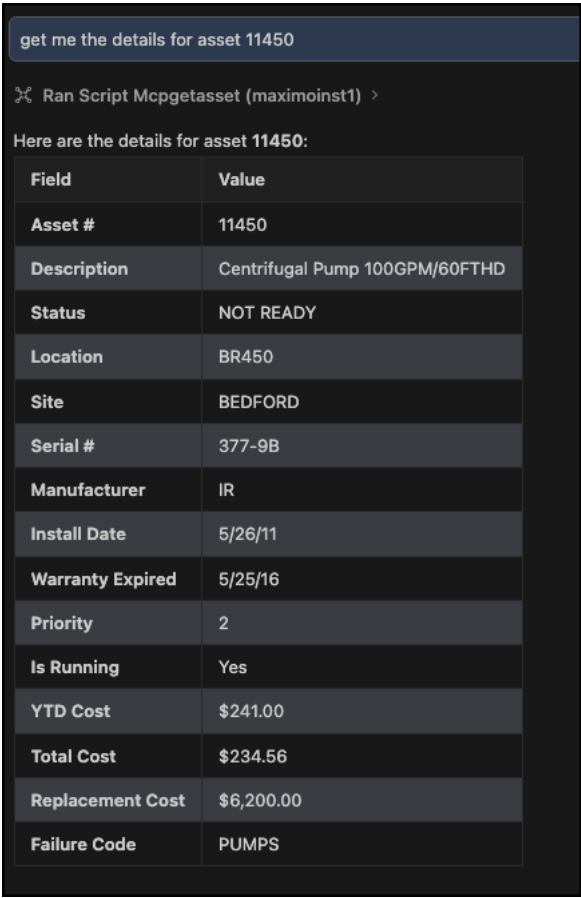
Exercise 10.3 — Look Up the Asset

Before raising records, retrieve the current asset information to confirm its details.

- 1. Ask Bob:
 “Get me the details for asset 11450.”
- 2. Bob will use the script mcpgetasset tool.



- 3. Click Approve once and Bob will fetch the data.



4. Confirm the following from the response:

Field	Expected
assetnum	11450
siteid	BEDFORD
isrunning	true or false
status	Shown
description	Shown
warrantyexpdate	Shown — note whether warranty is active or expired
ytdcost	Year-to-date maintenance cost
totalcost	Total historical maintenance cost

Exercise 10.4 — Look Up the PM Records

1. With the asset confirmed, check the PM schedule to understand whether maintenance is current or overdue. Ask Bob:
2. **“What preventive maintenance is scheduled for asset 11450? When was it last completed and when is it next due?”**
3. Bob will call `script_mcpgetpm` and return the PM records for the asset.
4. Click Approve once and Bob will attempt to fetch the data, however no records will be return using the `script_mcpgetpm` tool.
5. Bob will request approval to use the `restapi`. Approve the request, the rest call will fail as the MAXADMIN security group does not have READ access to the `mxapipm` object structure.
6. In Maximo in the Find Navigation Item enter Security.
7. Select Security→Security Groups (Manage)
8. In the name field enter MAXADMIN.
9. Select MAXADMIN then click the Object Structures tab.
10. Click the filter icon on the Object Structure table.

Group

Sites

Applications

Object Structures

Storerooms

Labor

GL Components

Limits and Tolerances

Data Restrict

Name

MAXADMIN

Maximo Administrators (Super Users)

Object Structures

(1 - 10 of 260)

Actions

Object Structure	Description
MXAPIAHCLASSIFICATN	AH Maximo API for Classification Definition
MXAPIAHMEASUREPOINT	AH Maximo API for Measurepoint
MXAPIASSETRRPLAN	AHASSETRRPLAN view Object Structure
MXAPIAUCAPAD	AUCAPAD Object Structure

11. Enter 'mxapipm' in the object structure name field and click enter.

Group

Sites

Applications

Object Structures

Storerooms

Labor

GL Components

Limits and Tolerances

Data Restrict

Name

MAXADMIN

Maximo Administrators (Super Users)

Object Structures

(1 - 1 of 1)

Actions

Object Structure	Description
mxapipm	
MXAPIPM	UI Preventive Maintenance Definition

1 - 1 of 1

12. In the Option for UI Preventive Maintenance Definition check the Read MXAPIPM checkbox under the Grant Access? column

Options for UI Preventive Maintenance Definition

(1 - 4 of 4)

Grant Listed Options for This Object Structure

	Description	Grant Access?	Condition
▼	Delete MXAPIPM	☐	>
▼	Insert MXAPIPM	☐	>
▼	Read MXAPIPM	☒	>
▼	Save MXAPIPM	☐	>

1 - 4 of 4

13. Save the record.

14. Logout of maximo, otherwise security will not get update.

15. Since there are no PM records for asset 11450 ask Bob?

“What preventive maintenance is scheduled for asset 11230? When was it last completed and when is it next due?”

NOTE: You may get comments such as we tried this earlier and it didn’t work. Tell Bob security has changed try again. If you are still getting the error logout of Maximo. If you are logged out, log back in and check to ensure you saved when you exited.

16. Bob will call `script_mcpgetpm` and return the PM records for the asset.

17. Click Approve once and Bob will fetch the data.

What preventive maintenance is scheduled for asset 11230? When was it last completed and when is it next due?

From the data already returned, I can see asset 11230 has a PM record:

Field	Value
PM #	PM-GEN
Description	Generator Overhaul
Asset	11230
Last Completed	2010-12-27
Next Due	N/A

The next due date is not set — which likely means the PM frequency hasn't been configured or the PM is inactive. Would you like me to fetch the full details?

Confirm the following from the response:

Field	Expected
pmnum	Shown
lastcomptime	Last PM completion date
nextdate	Next PM due date — note if overdue
frequency	Frequency value
frequnit	Frequency unit

Exercise 10.5 — Full Asset Condition and PM Analysis Report

In this exercise you combine every read-only tool built in this lab to produce a comprehensive condition and PM analysis report for asset 11450, then ask Bob to render it as a shareable HTML report artifact.

This exercise demonstrates Bob's ability to orchestrate multiple MCP tool calls, synthesise the results, identify risk findings, and produce a formatted deliverable — all from a single natural-language request.

1. Ask Bob:

“Perform a full analysis of asset 11450 including PM analysis and create an HTML report.”

2. Bob will:

3. Call `script_assetinformationquery` to retrieve asset details

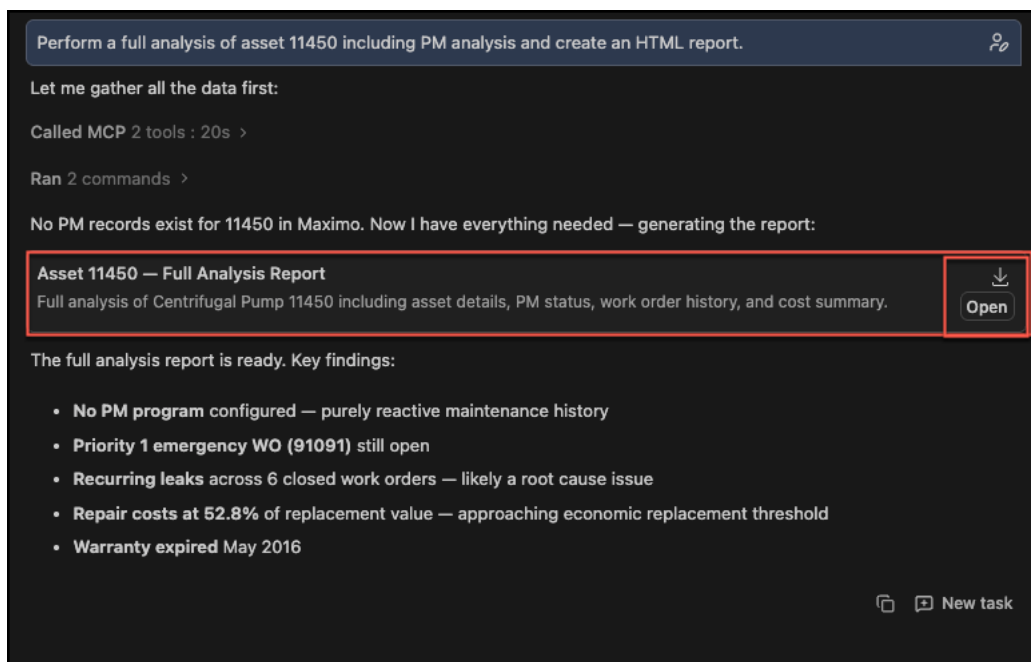
4. Call `script_mcpgetassetpm` to retrieve PM records by asset and location

5. Synthesise warranty status, cost data, operational flags, and PM coverage

6. Identify risk findings and recommended actions

7. Render the results as a self-contained HTML report artifact

8. Click the 'Open' button.



9. The report will open in the Tab view of the UI. Scroll through the Asset Analysis Report. The top header has a critical warning outlining key finding, Asset status, warranty expired, Emergency work orders still open.

Asset Analysis Report

Asset 11450 — Centrifugal Pump 100GPM/60FTHD — Generated from MAS-INST1 / BEDFORD

Critical: Asset status is **NOT READY** — asset is not currently operational despite *isRunning = true*. Warranty expired **5/25/2016**. Priority 1 emergency WO (91091) is open.

Warning: No preventive maintenance records configured for this asset. Reactive maintenance pattern observed across work order history.

10. The Asset Details section contains key identifying information.

Asset Details	
ASSET NUMBER 11450	STATUS NOT READY
DESCRIPTION Centrifugal Pump 100GPM/60FTHD	LOCATION BR450
SITE BEDFORD	MANUFACTURER IR
SERIAL NUMBER 377-9B	INSTALL DATE May 26, 2011
WARRANTY EXPIRED May 25, 2016	PRIORITY 2
FAILURE CODE PUMPS	REPLACEMENT COST \$6,200.00

11. The Cost Summary section identifies the asset related costs

Cost Summary

YTD COST \$241.00	TOTAL LIFETIME COST \$234.56
CLOSED WO LABOR COST \$2,749.25	CLOSED WO MATERIAL COST \$524.00
TOTAL CLOSED WO COST \$3,273.25	REPLACEMENT COST \$6,200.00

Cumulative repair costs represent 52.8% of replacement cost — approaching economic replacement threshold.

12. The Preventive Maintenance Analysis section would identify any PM work and recommendations.

Preventive Maintenance Analysis

**No PM Records Found:** Asset 11450 has no scheduled preventive maintenance configured in Maximo. The work order history shows two CM job plans exist (JP11430 — 12 Month, JP11430A — 3 Month) but neither is linked to an active PM schedule for this asset. Recommend creating PM records immediately.

PM #	DESCRIPTION	FREQUENCY	LAST COMPLETED	NEXT DUE	STATUS
------	-------------	-----------	----------------	----------	--------

No PM records configured for this asset

Recommended: Create PM using JP11430A (3-Month Service) and JP11430 (12-Month Service) for this asset.

13. Followed by the work order section.

Open Work Orders 26 open

WO #	DESCRIPTION	STATUS	PRIORITY	TYPE
91091	11450 BREAKDOWN	APPR	1	EM
1002	Rebuild Feedwater Pump	APPR	3	CM
1006	Feedwater Pump Service	APPR	7	CM
1201	Centrifugal Pump Service - 3 Month	WAPPR	8	—
1202	Lights flickering in room 3 near asset 11450	WAPPR	—	—
1203	Follow-up: Replace entire light fixture	WAPPR	—	—

+ 20 sub-tasks (1002-xx, 1006-xx, T1074–T1079)

Closed Work Order History 15 closed

WO #	DESCRIPTION	PRIORITY	TYPE	LABOR	MATERIALS
2433	Pump Stopped Running	1	EM	\$68.25	\$0.00
88051	11450 PROBLEM	1	EM	\$160.75	\$55.00
88111	LEAK	1	EM	\$125.75	\$40.00
89041	LEAK	1	EM	\$50.75	\$20.00
89081	11450 PROBLEM	1	EM	\$75.75	\$5.00
90061	LEAK	1	EM	\$90.00	\$22.00

14. Lastly the Recommendations section.

Recommendations

#	PRIORITY	RECOMMENDATION
1	Critical	Resolve open Priority 1 emergency WO 91091 (Breakdown) immediately
2	Critical	Create PM schedule using JP11430A (3-Month) and JP11430 (12-Month) — asset has no preventive maintenance program
3	High	Review and approve pending WOs 1201, 1202, 1203 (WAPPR status)
4	High	Warranty expired May 2016 — evaluate extended service contract or replacement plan
5	Medium	Recurring leak pattern across 6 closed WOs — investigate root cause (seals, housing, connections)
6	Medium	Total repair cost (\$3,273) = 52.8% of replacement cost (\$6,200) — assess replacement vs. continued repair

Made with IBM Bob